

Vorabfragen

- Sie suchen in einem sortierten Array mit 1.000.000 Elementen einen Wert, der gar nicht vorhanden ist. Mit der Strategie „Prüfe die Mitte und schließe dann eine Hälfte aus“ (Binäre Suche): Wie viele Elemente müssen Sie im Worst Case ungefähr untersuchen?
 - A) ca. 20; B) ca. 1.000; C) ca. 500.000; D) ca. 1.000.000
- Mergesort halbiert die Eingabe in jedem Schritt, genau wie die binäre Suche. Trotzdem ist seine Laufzeit nicht $\Theta(\log n)$, sondern $\Theta(n \log n)$. Woher könnte der zusätzliche Aufwand kommen?
- Die Fibonacci-Zahl lässt sich rekursiv berechnen: $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$. Angenommen, $\text{fib}(40)$ braucht 1 Sekunde. Wie lange dauert $\text{fib}(41)$ etwa, wie lange $\text{fib}(50)$? Begründen Sie kurz Ihre Schätzung.



Algorithmik – WS 26/27

Gelesen von Prof. Dr. Daniel Gaida

04.10.2026

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Technology
Arts Sciences
TH Köln

Lernraum I: Abstrakte Datentypen, Datenstrukturen und Asymptotische Notation

- Abstrakte Datentypen
 - Liste, Stapel, Warteschlange, Tabelle (Dictionary)
- Datenstrukturen
 - Array, Verkettete Liste
- -
 -
- -
- -
- -
 -
-

Lernraum I: Abstrakte Datentypen, Datenstrukturen und Asymptotische Notation

- Abstrakte Datentypen
 - Liste, Stapel, Warteschlange, Tabelle (Dictionary)
- Datenstrukturen
 - Array, Verkettete Liste
- Implementierungen der Abstrakten Datentypen mit den Datenstrukturen ...
 - Array
 - Verkettete Listen
- Analyse von Laufzeit und Speicherbedarf der Implementierungen
 - O-Notation
- -
 -
-

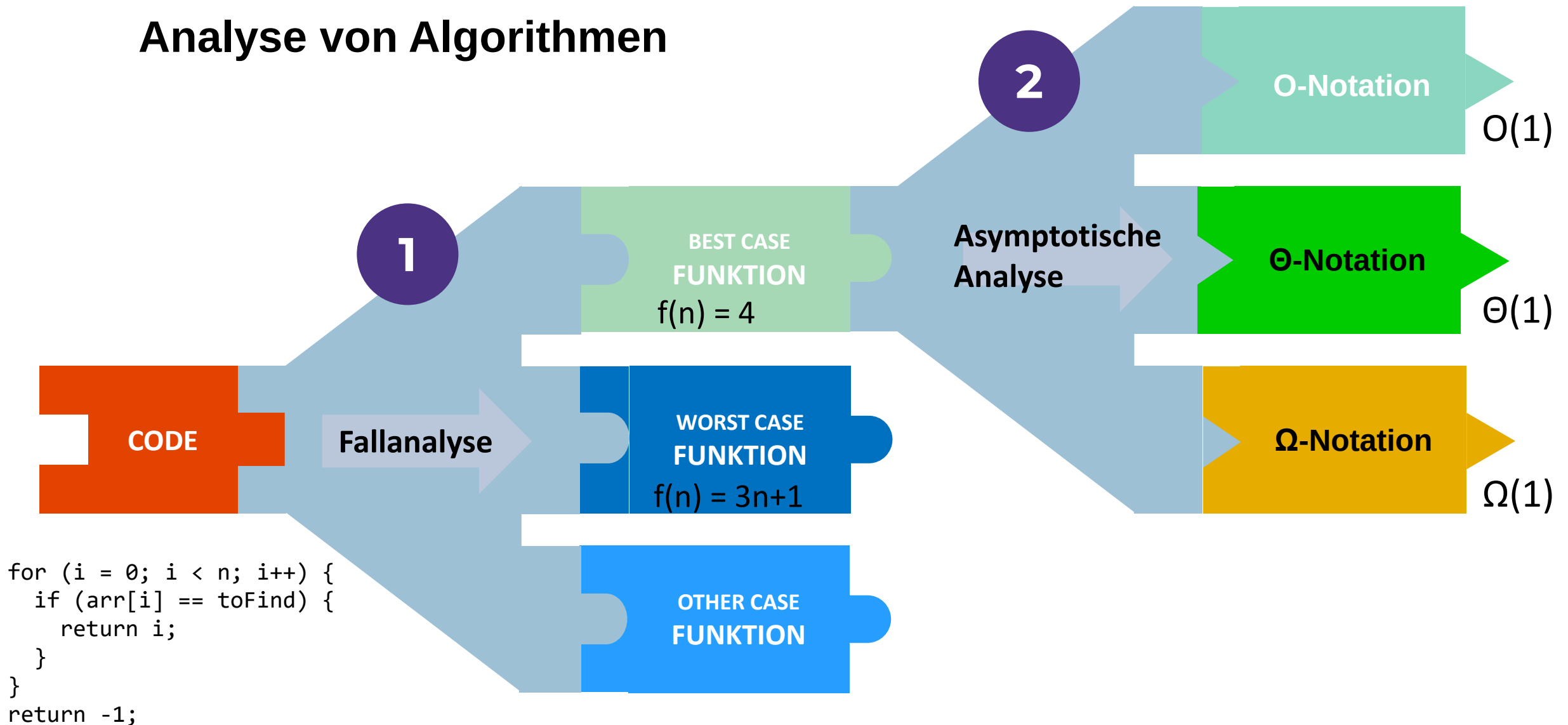
Lernraum I: Abstrakte Datentypen, Datenstrukturen und Asymptotische Notation

- Abstrakte Datentypen
 - Liste, Stapel, Warteschlange, Tabelle (Dictionary)
- Datenstrukturen
 - Array, Verkettete Liste
- Implementierungen der Abstrakten Datentypen mit den Datenstrukturen ...
 - Array
 - Verkettete Listen
- Analyse von Laufzeit und Speicherbedarf der Implementierungen
 - O-Notation
- Laufzeitanalyse von Algorithmen durch Asymptotische Notation
 - O-Notation, Ω -Notation, Θ -Notation
 - Fallanalyse: Worst Case, Best Case Analyse
-

Lernraum I: Abstrakte Datentypen, Datenstrukturen und Asymptotische Notation

- Abstrakte Datentypen
 - Liste, Stapel, Warteschlange, Tabelle (Dictionary)
- Datenstrukturen
 - Array, Verkettete Liste
- Implementierungen der Abstrakten Datentypen mit den Datenstrukturen ...
 - Array
 - Verkettete Listen
- Analyse von Laufzeit und Speicherbedarf der Implementierungen
 - O-Notation
- Laufzeitanalyse von Algorithmen durch Asymptotische Notation
 - O-Notation, Ω -Notation, Θ -Notation
 - Fallanalyse: Worst Case, Best Case Analyse
- Laufzeitanalyse von Rekursiven Algorithmen durch Asymptotische Notation

Analyse von Algorithmen



Laufzeitanalyse von Rekursiven Algorithmen durch Asymptotische Notation

Bei der Modellierung und Analyse von rekursivem Code geht es darum, Muster zu finden, die beschreiben wie sich die Eingabe zwischen den Aufrufen ändert und wie viel Arbeit bei jedem Aufruf geleistet wird.

-
-
-

Laufzeitanalyse von Rekursiven Algorithmen durch Asymptotische Notation

Bei der Modellierung und Analyse von rekursivem Code geht es darum, Muster zu finden, die beschreiben wie sich die Eingabe zwischen den Aufrufen ändert und wie viel Arbeit bei jedem Aufruf geleistet wird.

Lassen Sie uns einige der häufigsten rekursiven Muster untersuchen

- **Muster Nr. 1:** Halbierung der Eingabe
- **Muster Nr. 2:** Teilen der Eingabe, Beherrschen und Vereinigen (Teile-und-Beherrsche)
- **Muster Nr. 3:** Verdoppelung der Eingabe

Beispiel für Muster 1: Binäre Suche

```
public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}
```



Annahme: arr ist
aufsteigend sortiert

Beispiel für Muster 1: Binäre Suche

Laufzeit

Binäre Suche: Findet einen Zielwert in einem sortierten Array, indem sie nacheinander die Hälfte des Arrays aus der Betrachtung ausschließt.

Beispiel: Durchsuchen des unten stehenden Arrays nach dem Wert 42:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	-4	2	7	10	15	20	22	25	30	36	42	50	56	68	85	92	103



Beispiel für Muster 1: Binäre Suche

Laufzeit

Binäre Suche: Findet einen Zielwert in einem sortierten Array, indem sie nacheinander die Hälfte des Arrays aus der Betrachtung ausschließt.

Beispiel: Durchsuchen des unten stehenden Arrays nach dem Wert 42:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	-4	2	7	10	15	20	22	25	30	36	42	50	56	68	85	92	103

lo

mid

hi

Beispiel für Muster 1: Binäre Suche

Laufzeit

Binäre Suche: Findet einen Zielwert in einem sortierten Array, indem sie nacheinander die Hälfte des Arrays aus der Betrachtung ausschließt.

Beispiel: Durchsuchen des unten stehenden Arrays nach dem Wert 42:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	-4	2	7	10	15	20	22	25	30	36	42	50	56	68	85	92	103

lo

mid

hi

Beispiel für Muster 1: Binäre Suche

Laufzeit

Binäre Suche: Findet einen Zielwert in einem sortierten Array, indem sie nacheinander die Hälfte des Arrays aus der Betrachtung ausschließt.

Beispiel: Durchsuchen des unten stehenden Arrays nach dem Wert 42:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	-4	2	7	10	15	20	22	25	30	36	42	50	56	68	85	92	103

↑
lo

↑
mid

↑
hi

Beispiel für Muster 1: Binäre Suche

Laufzeit

Binäre Suche: Findet einen Zielwert in einem sortierten Array, indem sie nacheinander die Hälfte des Arrays aus der Betrachtung ausschließt.

Beispiel: Durchsuchen des unten stehenden Arrays nach dem Wert 42:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	-4	2	7	10	15	20	22	25	30	36	42	50	56	68	85	92	103

lo

36

mid

56

hi

103



Beispiel für Muster 1: Binäre Suche

Laufzeit

Binäre Suche: Findet einen Zielwert in einem sortierten Array, indem sie nacheinander die Hälfte des Arrays aus der Betrachtung ausschließt.

Beispiel: Durchsuchen des unten stehenden Arrays nach dem Wert 42:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	-4	2	7	10	15	20	22	25	30	36	42	50	56	68	85	92	103



Beispiel für Muster 1: Binäre Suche

Laufzeit

Binäre Suche: Findet einen Zielwert in einem sortierten Array, indem sie nacheinander die Hälfte des Arrays aus der Betrachtung ausschließt.

Beispiel: Durchsuchen des unten stehenden Arrays nach dem Wert 42:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	-4	2	7	10	15	20	22	25	30	36	42	50	56	68	85	92	103

lo

mid

hi



Beispiel für Muster 1: Binäre Suche

Laufzeit

Binäre Suche: Findet einen Zielwert in einem sortierten Array, indem sie nacheinander die Hälfte des Arrays aus der Betrachtung ausschließt.

Beispiel: Durchsuchen des unten stehenden Arrays nach dem Wert 42:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	-4	2	7	10	15	20	22	25	30	36	42	50	56	68	85	92	103

lo

mid

hi

Wie viele Elemente werden untersucht?

- Was ist der Best Case?

■

■

■

Beispiel für Muster 1: Binäre Suche

Laufzeit

Binäre Suche: Findet einen Zielwert in einem sortierten Array, indem sie nacheinander die Hälfte des Arrays aus der Betrachtung ausschließt.

Beispiel: Durchsuchen des unten stehenden Arrays nach dem Wert 42:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	-4	2	7	10	15	20	22	25	30	36	42	50	56	68	85	92	103

lo

mid

hi

Wie viele Elemente werden untersucht?

- Was ist der Best Case?
 - Element gefunden bei Index 8, 1 Element untersucht, $O(1)$
- Was ist der Worst Case?
-

Beispiel für Muster 1: Binäre Suche

Laufzeit

Binäre Suche: Findet einen Zielwert in einem sortierten Array, indem sie nacheinander die Hälfte des Arrays aus der Betrachtung ausschließt.

Beispiel: Durchsuchen des unten stehenden Arrays nach dem Wert 42:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	-4	2	7	10	15	20	22	25	30	36	42	50	56	68	85	92	103

lo

mid

hi

Wie viele Elemente werden untersucht?

- Was ist der Best Case?
 - Element gefunden bei Index 8, 1 Element untersucht, $O(1)$
- Was ist der Worst Case?
 - Element nicht gefunden, $\frac{1}{2}$ der Elemente untersucht, dann $\frac{1}{2}$ davon...

Modellierung von rekursivem Code

```
public int binarySearch(int[] arr, int toFind, int lo, int hi) {  
    if( hi < lo ) {  
        return -1;  
    } if(hi == lo) {  
        if(arr[hi] == toFind) {  
            return hi;  
        }  
        return -1;  
    }  
    int mid = (lo+hi) / 2;  
    if(arr[mid] == toFind) {  
        return mid;  
    } else if(arr[mid] < toFind) {  
        return binarySearch(arr, toFind, mid+1, hi);  
    } else {  
        return binarySearch(arr, toFind, lo, mid-1);  
    }  
}
```

Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}

```

Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) { 
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}

```

Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}

```

Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}

```

Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}

```

Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {  1+1
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {  3
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}

```

Diagram illustrating the complexity analysis of the binarySearch function using recursive calls and base cases:

- The base case `if( hi < lo )` is annotated with a brace and the value **1+1**.
- The recursive case `if(hi == lo)` is annotated with a brace and the value **1+3**.
- The recursive call `binarySearch(arr, toFind, mid+1, hi)` is annotated with a brace and the value **3**.

Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}

```

Complexity annotations for the binarySearch function:

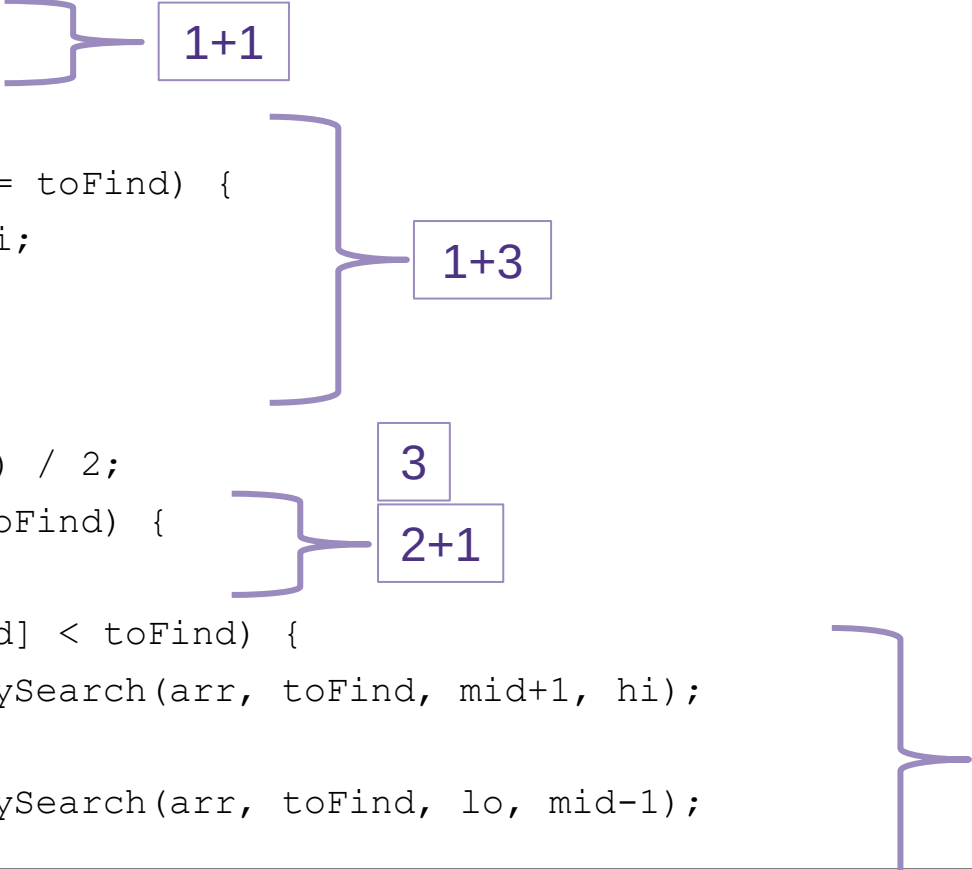
- Base case (`if( hi < lo )`): $1+1$
- Recursive case (`if(hi == lo)`): $1+3$
- Mid calculation (`int mid = (lo+hi) / 2;`): 3
- Recursive call (`return binarySearch(arr, toFind, mid+1, hi);`): $2+1$

Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {  1+1
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {  3
        return mid;  2+1
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}

```



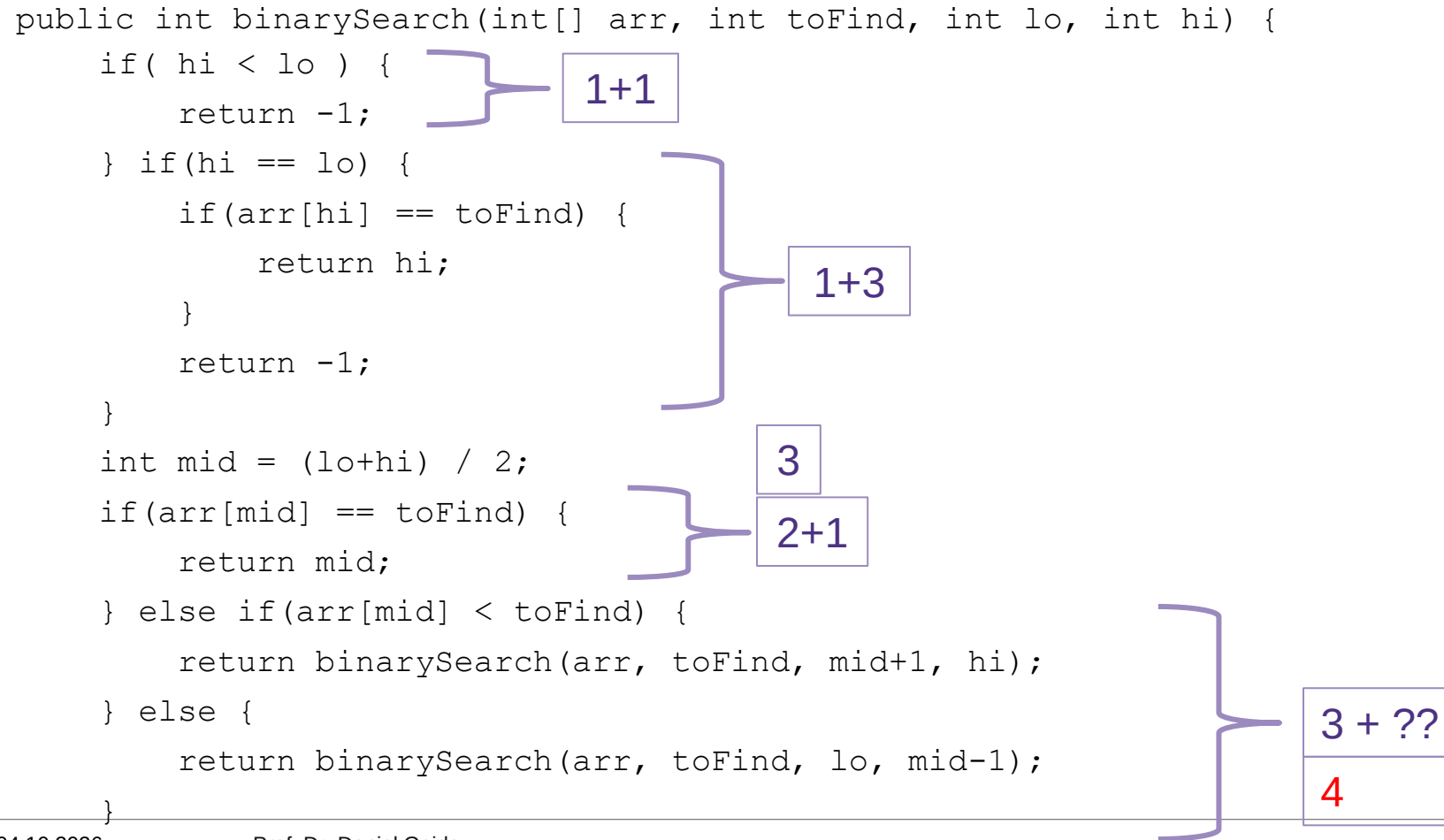
Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}

```

Modellierung von rekursivem Code



Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}

```

1+1

1+3

3

2+1

3 + ??

4

Wie modelliert man
rekursive Aufrufe?

Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}
    
```

1+1

1+3

3

2+1

3 + ??

4

Wie modelliert man
rekursive Aufrufe?
Mit einer
Rekursionsgleichung!

Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}
    
```

1+1

1+3

3

2+1

$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

3 + ??

4

Wie modelliert man
rekursive Aufrufe?
Mit einer
Rekursionsgleichung!

Rekursionsgleichung

Eine Rekursionsgleichung ist eine Gleichung, die eine Funktion durch ihre eigenen Funktionswerte für kleinere Eingaben beschreibt



$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

Rekursionsgleichung

Eine Rekursionsgleichung ist eine Gleichung, die eine Funktion durch ihre eigenen Funktionswerte für kleinere Eingaben beschreibt

Mit Rekursionsgleichungen wird rekursiver Code beschrieben:

- Mindestens ein Basisfall und mindestens ein rekursiver Fall
-
-
-

$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

Rekursionsgleichung

Eine Rekursionsgleichung ist eine Gleichung, die eine Funktion durch ihre eigenen Funktionswerte für kleinere Eingaben beschreibt

Mit Rekursionsgleichungen wird rekursiver Code beschrieben:

- Mindestens ein Basisfall und mindestens ein rekursiver Fall
 - Basisfall bei binärer Suche: Nur noch ein Element zu durchsuchen
-
-

$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

Rekursionsgleichung

Eine Rekursionsgleichung ist eine Gleichung, die eine Funktion durch ihre eigenen Funktionswerte für kleinere Eingaben beschreibt

Mit Rekursionsgleichungen wird rekursiver Code beschrieben:

- Mindestens ein Basisfall und mindestens ein rekursiver Fall
 - Basisfall bei binärer Suche: Nur noch ein Element zu durchsuchen
- Der rekursive Fall sollte die Eingabegröße so reduzieren, dass schließlich der Basisfall ausgelöst wird.
- Die Fälle der Rekursionsgleichung entsprechen normalerweise genau den Fällen des Codes

$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}

```

1+1

1+3

3

2+1

$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

3 + ??

Wie modelliert man
rekursive Aufrufe?
Mit einer
Rekursionsgleichung!

Modellierung von rekursivem Code

```

public int binarySearch(int[] arr, int toFind, int lo, int hi) {
    if( hi < lo ) {
        return -1;
    } if(hi == lo) {
        if(arr[hi] == toFind) {
            return hi;
        }
        return -1;
    }
    int mid = (lo+hi) / 2;
    if(arr[mid] == toFind) {
        return mid;
    } else if(arr[mid] < toFind) {
        return binarySearch(arr, toFind, mid+1, hi);
    } else {
        return binarySearch(arr, toFind, lo, mid-1);
    }
}

```

1+1

1+3

3

2+1

$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

1+1+3=5

1+1+3+2+3=10

3 + ??

Wie modelliert man
rekursive Aufrufe?

Mit einer
Rekursionsgleichung!

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

Schritt 1: Laufzeitfunktion bestimmen. OK!

Schritt 2: Asymptotische Analyse durchführen.

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

Schritt 1: Laufzeitfunktion bestimmen. OK!

Schritt 2: Asymptotische Analyse durchführen.

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

Schritt 1: Laufzeitfunktion bestimmen. OK!

Schritt 2: Asymptotische Analyse durchführen.

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

$$a = 1$$

$$b = 2$$

$$c = 0$$

$$d = 5$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

Mastermethode angewandt auf Binäre Suche

Übung

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

$$a = 1, b = 2, c = 0, d = 5$$

$$f(n) \in \Theta(n^0) = \Theta(1)$$

Mastermethode angewandt auf Binäre Suche

Übung

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

$$a = 1, b = 2, c = 0, d = 5$$

$$f(n) \in \Theta(n^0) = \Theta(1)$$

$$x = \log_b y \text{ entspricht } b^x = y$$

$$\log_2 1 = x \implies 2^x = 1 \implies x = 0$$

Mastermethode angewandt auf Binäre Suche

Übung

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

$$T(n) = \begin{cases} 5 & \text{falls } n < 2 \\ T\left(\frac{n}{2}\right) + 10 & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$a = 1, b = 2, c = 0, d = 5$$

$$f(n) \in \Theta(n^0) = \Theta(1)$$

$$x = \log_b y \text{ entspricht } b^x = y$$

$$\log_2 1 = x \implies 2^x = 1 \implies x = 0$$

$$\log_2 1 = 0$$

Wir sind in Fall 2: $T(n) \in \Theta(1 \log n) = \Theta(\log n)$

Eine Rekursionsgleichung herleiten

Übung

```
public int recursiveFunction(int n) {  
    if(n < 3) {  
        return 3;  
    }  
    for(int i=0; i < n; i++) {  
        System.out.println(i);  
    }  
    int val1 = recursiveFunction(n/3);  
    int val2 = recursiveFunction(n/3);  
    return val1 * val2;  
}
```

Eine Rekursionsgleichung herleiten

Übung

```
public int recursiveFunction(int n){  
    if(n < 3) {  
        return 3;  
    }  
    for(int i=0; i < n; i++) {  
        System.out.println(i);  
    }  
    int val1 = recursiveFunction(n/3);  
    int val2 = recursiveFunction(n/3);  
    return val1 * val2;  
}
```

Basisfall

Rekursiver Fall
Nicht-rekursive Arbeit
Rekursive Arbeit

Eine Rekursionsgleichung herleiten

Übung

```
public int recursiveFunction(int n){
    if(n < 3) {
        return 3;
    }
    for(int i=0; i < n; i++) {
        System.out.println(i);
    }
    int val1 = recursiveFunction(n/3);
    int val2 = recursiveFunction(n/3);
    return val1 * val2;
}
```

Basisfall 1+1

Rekursiver Fall
Nicht-rekursive Arbeit
Rekursive Arbeit

Eine Rekursionsgleichung herleiten

Übung

```
public int recursiveFunction(int n){
```

```
    if(n < 3) {
```

```
        return 3;
```

```
    }
```

```
    for(int i=0; i < n; i++) {
```

```
        System.out.println(i);
```

```
    }
```

```
    int val1 = recursiveFunction(n/3);
```

```
    int val2 = recursiveFunction(n/3);
```

```
    return val1 * val2;
```

```
}
```

Basisfall 1+1

+n

+2

+2

+2

Rekursiver Fall
Nicht-rekursive Arbeit
Rekursive Arbeit

Eine Rekursionsgleichung herleiten

Übung

```
public int recursiveFunction(int n){
```

```
    if(n < 3) {
```

```
        return 3;
```

```
    }
```

```
    for(int i=0; i < n; i++) {
```

```
        System.out.println(i);
```

```
    }
```

```
    int val1 = recursiveFunction(n/3);
```

```
    int val2 = recursiveFunction(n/3);
```

```
    return val1 * val2;
```

```
}
```

Basisfall 1+1

+n

+2

+2

+2

Rekursiver Fall

Nicht-rekursive Arbeit

n+6

Rekursive Arbeit

Eine Rekursionsgleichung herleiten

Übung

```
public int recursiveFunction(int n){
```

```
    if(n < 3) {
```

```
        return 3;
```

```
    }
```

```
    for(int i=0; i < n; i++) {
```

```
        System.out.println(i);
```

```
    }
```

```
    int val1 = recursiveFunction(n/3);
```

```
    int val2 = recursiveFunction(n/3);
```

```
    return val1 * val2;
```

```
}
```

Basisfall 1+1

+n

+2

+2

+2

Rekursiver Fall

Nicht-rekursive Arbeit

n+6

Rekursive Arbeit

2*T(n/3)

Eine Rekursionsgleichung herleiten

Übung

```
public int recursiveFunction(int n){
```

```
    if(n < 3) {
```

```
        return 3;
```

```
    }
```

```
    for(int i=0; i < n; i++) {
```

```
        System.out.println(i);
```

```
    }
```

```
    int val1 = recursiveFunction(n/3);
```

```
    int val2 = recursiveFunction(n/3);
```

```
    return val1 * val2;
```

```
}
```

Basisfall 1+1

+n

+2

+2

+2

Rekursiver Fall

Nicht-rekursive Arbeit

n+6

Rekursive Arbeit

2*T(n/3)

$$T(n) = \begin{cases} 2 & \text{falls } n < 3 \\ 2T\left(\frac{n}{3}\right) + n + 7 & \text{sonst} \end{cases}$$

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 2 & \text{falls } n < 3 \\ 2T\left(\frac{n}{3}\right) + n & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 2 & \text{falls } n < 3 \\ 2T\left(\frac{n}{3}\right) + n & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$a = 2, b = 3, c = 1$$

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 2 & \text{falls } n < 3 \\ 2T\left(\frac{n}{3}\right) + n & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$a = 2, b = 3, c = 1$$

$$x = \log_b y \text{ entspricht } b^x = y$$

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 2 & \text{falls } n < 3 \\ 2T\left(\frac{n}{3}\right) + n & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$a = 2, b = 3, c = 1$$

$$x = \log_b y \text{ entspricht } b^x = y$$

$$\log_3 2 = x \implies 3^x = 2 \implies x \approx 0.63$$

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 2 & \text{falls } n < 3 \\ 2T\left(\frac{n}{3}\right) + n & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$a = 2, b = 3, c = 1$$

$$x = \log_b y \text{ entspricht } b^x = y$$

$$\log_3 2 = x \implies 3^x = 2 \implies x \approx 0.63$$

$$\log_3 2 < 1$$

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 2 & \text{falls } n < 3 \\ 2T\left(\frac{n}{3}\right) + n & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$a = 2, b = 3, c = 1$$

$$x = \log_b y \text{ entspricht } b^x = y$$

$$\log_3 2 = x \implies 3^x = 2 \implies x \approx 0.63$$

$$\log_3 2 < 1$$

Wir sind in Fall 1: $T(n) \in \Theta(n)$

Die Mastermethode verstehen

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

- a misst, wie viele rekursive Aufrufe in jeder Methodeninstanz ausgeführt werden

-
-

-

Die Mastermethode verstehen

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

- a misst, wie viele rekursive Aufrufe in jeder Methodeninstanz ausgeführt werden
- b misst die Änderungsrate der Eingabe
-
-

Die Mastermethode verstehen

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

- a misst, wie viele rekursive Aufrufe in jeder Methodeninstanz ausgeführt werden
- b misst die Änderungsrate der Eingabe
- c misst den dominierenden Term des nicht-rekursiven Code innerhalb der rekursiven Methode
-

Die Mastermethode verstehen

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

- a misst, wie viele rekursive Aufrufe in jeder Methodeninstanz ausgeführt werden
- b misst die Änderungsrate der Eingabe
- c misst den dominierenden Term des nicht-rekursiven Code innerhalb der rekursiven Methode
- d misst die Laufzeit des Basisfalls

Die Mastermethode verstehen

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

- a misst, wie viele rekursive Aufrufe in jeder Methodeninstanz ausgeführt werden
- b misst die Änderungsrate der Eingabe
- c misst den dominierenden Term des nicht-rekursiven Code innerhalb der rekursiven Methode
- d misst die Laufzeit des Basisfalls

Übungsblatt 3: Aufgabe 4

Die Mastermethode verstehen

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

Der Fall „ $\log_b a < c$ “

- Der rekursive Fall macht eine Menge nicht-rekursiver Arbeit im Vergleich dazu, wie viel Arbeit an nachfolgende rekursive Aufrufe weitergegeben wird

▪

▪

Die Mastermethode verstehen

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

Der Fall „ $\log_b a < c$ “

- Der rekursive Fall macht eine Menge nicht-rekursiver Arbeit im Vergleich dazu, wie viel Arbeit an nachfolgende rekursive Aufrufe weitergegeben wird
- Nicht-rekursive Arbeit im rekursiven Fall dominiert das Wachstum, n^c -Term
- Äquivalent: $a < b^c$

Die Mastermethode verstehen

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

Der Fall „ $\log_b a = c$ “

- Im rekursiven Fall wird die Arbeit gleichmäßig zwischen nicht-rekursiver Arbeit und der Weitergabe von Eingaben an nachfolgende rekursive Aufrufe aufgeteilt

Die Mastermethode verstehen

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

Der Fall „ $\log_b a = c$ “

- Im rekursiven Fall wird die Arbeit gleichmäßig zwischen nicht-rekursiver Arbeit und der Weitergabe von Eingaben an nachfolgende rekursive Aufrufe aufgeteilt

Der Fall „ $\log_b a > c$ “

- Der rekursive Fall gibt Arbeit größtenteils an nachfolgende rekursive Aufrufe weiter und macht nicht viel nicht-rekursive Arbeit

Eine Rekursionsgleichung herleiten

Beispiel für Fall 3

```
public int recursiveFunction(int n){
    if(n < 3) {
        return 3;
    }
    for(int i=0; i < n; i++) {
        System.out.println(i);
    }
    int val1 = recursiveFunction(n/3);
    int val2 = recursiveFunction(n/3);
    return val1 * val2;
}
```

Eine Rekursionsgleichung herleiten

Beispiel für Fall 3

```
public int recursiveFunction(int n){
    if(n < 3) {
        return 3;
    }

    for(int i=0; i < n; i++){
        System.out.println(i);
    }

    int val1 = recursiveFunction(n/3);
    int val2 = recursiveFunction(n/3);
    return val1 * val2;
}
```

Basisfall

Rekursiver Fall
Nicht-rekursive Arbeit
Rekursive Arbeit

Eine Rekursionsgleichung herleiten

Beispiel für Fall 3

```
public int recursiveFunction(int n){
```

```
    if(n < 3) {
```

```
        return 3;
```

```
    }
```

Basisfall 1+1

```
    for(int i=0; i < n; i++){
```

```
    System.out.println(i);
```

```
    }
```

```
    int val1 = recursiveFunction(n/3); +2
```

```
    int val2 = recursiveFunction(n/3); +2
```

```
    return val1 * val2; +2
```

```
}
```

Rekursiver Fall
Nicht-rekursive Arbeit
Rekursive Arbeit

Eine Rekursionsgleichung herleiten

Beispiel für Fall 3

```
public int recursiveFunction(int n){
```

```
    if(n < 3) {
```

```
        return 3;
```

```
    }
```

Basisfall 1+1

```
    for(int i=0; i < n; i++) {
```

```
    System.out.println(i);
```

```
    }
```

```
    int val1 = recursiveFunction(n/3); +2
```

```
    int val2 = recursiveFunction(n/3); +2
```

```
    return val1 * val2; +2
```

```
}
```

Rekursiver Fall

Nicht-rekursive Arbeit

6

Rekursive Arbeit

Eine Rekursionsgleichung herleiten

Beispiel für Fall 3

```
public int recursiveFunction(int n){
```

```
    if(n < 3) {
```

```
        return 3;
```

```
    }
```

```
    for(int i=0; i < n; i++){
```

```
            System.out.println(i);
```

```
    }
```

```
    int val1 = recursiveFunction(n/3);
```

+2

```
    int val2 = recursiveFunction(n/3);
```

+2

```
    return val1 * val2;
```

+2

```
}
```

Basisfall **1+1**

Rekursiver Fall

Nicht-rekursive Arbeit

6

Rekursive Arbeit

2*T(n/3)

Eine Rekursionsgleichung herleiten

Beispiel für Fall 3

```
public int recursiveFunction(int n){
```

```
    if(n < 3) {
```

```
        return 3;
```

```
    }
```

```
    for(int i=0; i < n; i++){
```

```
    System.out.println(i);
```

```
    }
```

```
    int val1 = recursiveFunction(n/3);
```

+2

```
    int val2 = recursiveFunction(n/3);
```

+2

```
    return val1 * val2;
```

+2

```
}
```

Basisfall **1+1**

Rekursiver Fall

Nicht-rekursive Arbeit

6

Rekursive Arbeit

2*T(n/3)

$$T(n) = \begin{cases} 2 & \text{falls } n < 3 \\ 2T\left(\frac{n}{3}\right) + 7 & \text{sonst} \end{cases}$$

Eine Rekursionsgleichung herleiten

Beispiel für Fall 3

```
public int recursiveFunction(int n) {
```

```
    if(n < 3) {
```

```
        return 3;
```

```
    }
```

```
    for(int i=0; i < n; i++) {
```

```
            System.out.println(i);
```

```
    }
```

```
    int val1 = recursiveFunction(n/3);
```

+2

```
    int val2 = recursiveFunction(n/3);
```

+2

```
    return val1 * val2;
```

+2

```
}
```

Basisfall **1+1**

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

Rekursiver Fall

Nicht-rekursive Arbeit

6

Rekursive Arbeit

2*T(n/3)

$$T(n) = \begin{cases} 2 & \text{falls } n < 3 \\ 2T\left(\frac{n}{3}\right) + 7 & \text{sonst} \end{cases}$$

Beispiele für Algorithmen mit asymptotischer Laufzeit mit „krummen“ Exponenten

Die naive Matrixmultiplikation hat eine Laufzeit von $O(n^3)$, da für zwei $n \times n$ -Matrizen im Allgemeinen n^3 Multiplikationen erforderlich sind (zur Berechnung jeder Zahl n Multiplikationen, da wir n^2 Zahlen berechnen müssen: n^3).

$$\begin{array}{ccc}
 \begin{array}{c} i \\ \begin{array}{|c|c|c|} \hline a_{11} & a_{12} & a_{13} \\ \hline a_{21} & a_{22} & a_{23} \\ \hline a_{31} & a_{32} & a_{33} \\ \hline \end{array} \end{array} & \times & \begin{array}{c} j \\ \begin{array}{|c|c|c|} \hline b_{11} & b_{12} & b_{13} \\ \hline b_{21} & b_{22} & b_{23} \\ \hline b_{31} & b_{32} & b_{33} \\ \hline \end{array} \end{array} \\
 A & \times & B \\
 \end{array} = \begin{array}{|c|c|c|} \hline c_{11} & c_{12} & c_{13} \\ \hline c_{21} & c_{22} & c_{23} \\ \hline c_{31} & c_{32} & c_{33} \\ \hline \end{array} = C$$

-
-
-

Beispiele für Algorithmen mit asymptotischer Laufzeit mit „krummen“ Exponenten

Die naive Matrixmultiplikation hat eine Laufzeit von $O(n^3)$, da für zwei $n \times n$ -Matrizen im Allgemeinen n^3 Multiplikationen erforderlich sind (zur Berechnung jeder Zahl n Multiplikationen, da wir n^2 Zahlen berechnen müssen: n^3).

$$\begin{array}{ccc}
 \begin{array}{c} i \\ \begin{array}{|c|c|c|} \hline a_{11} & a_{12} & a_{13} \\ \hline a_{21} & a_{22} & a_{23} \\ \hline a_{31} & a_{32} & a_{33} \\ \hline \end{array} \end{array} & \times & \begin{array}{c} j \\ \begin{array}{|c|c|c|} \hline b_{11} & b_{12} & b_{13} \\ \hline b_{21} & b_{22} & b_{23} \\ \hline b_{31} & b_{32} & b_{33} \\ \hline \end{array} \end{array} \\
 A & \times & B
 \end{array}
 =
 \begin{array}{|c|c|c|} \hline c_{11} & c_{12} & c_{13} \\ \hline c_{21} & c_{22} & c_{23} \\ \hline c_{31} & c_{32} & c_{33} \\ \hline \end{array}
 = C$$

Durch verschiedene Algorithmen wurde diese Laufzeit sukzessive verbessert:

- **Strassen-Algorithmus** (1969): Laufzeit verbessert auf $O(n^{\log_2(7)})$, also etwa $O(n^{2.81})$.
- **Coppersmith-Winograd-Algorithmus** (1987): Laufzeit von $O(n^{2.376})$.
- Algorithmus von **V. V. Williams** (2012): Laufzeit von $O(n^{2.3727})$.

Beispiele für Algorithmen mit asymptotischer Laufzeit mit „krummen“ Exponenten

Die naive Matrixmultiplikation hat eine Laufzeit von $O(n^3)$, da für zwei $n \times n$ -Matrizen im Allgemeinen n^3 Multiplikationen erforderlich sind (zur Berechnung jeder Zahl n Multiplikationen, da wir n^2 Zahlen berechnen müssen: n^3).

$$\begin{array}{ccc}
 \begin{array}{|c|c|c|} \hline a_{11} & a_{12} & a_{13} \\ \hline a_{21} & a_{22} & a_{23} \\ \hline a_{31} & a_{32} & a_{33} \\ \hline \end{array} & \times & \begin{array}{|c|c|c|} \hline b_{11} & b_{12} & b_{13} \\ \hline b_{21} & b_{22} & b_{23} \\ \hline b_{31} & b_{32} & b_{33} \\ \hline \end{array} \\
 i & & j \\
 A & \times & B \\
 \end{array} = \begin{array}{|c|c|c|} \hline c_{11} & c_{12} & c_{13} \\ \hline c_{21} & c_{22} & c_{23} \\ \hline c_{31} & c_{32} & c_{33} \\ \hline \end{array} = C$$

Durch verschiedene Algorithmen wurde diese Laufzeit sukzessive verbessert:

- **Strassen-Algorithmus** (1969): Laufzeit verbessert auf $O(n^{\log_2(7)})$, also etwa $O(n^{2.81})$.
- **Coppersmith-Winograd-Algorithmus** (1987): Laufzeit von $O(n^{2.376})$.
- Algorithmus von **V. V. Williams** (2012): Laufzeit von $O(n^{2.3727})$.

Diese Exponenten kommen durch die Nutzung rekursiver Algorithmen zustande.

Fragen?

- Modellierung von rekursivem Code
 - Mastermethode
-
- Übungsblatt 3 – Aufgabe 6

Code Modellierung: Rekursive Muster

Muster Nr. 1: Halbierung der Eingabe

- Binäre Suche – Laufzeit $\Theta(\log n)$

Muster Nr. 2: Teilen der Eingabe, Beherrschen und Vereinigen (Teile-und-Beherrsche)

- Mergesort

Muster Nr. 3: Verdoppelung der Eingabe

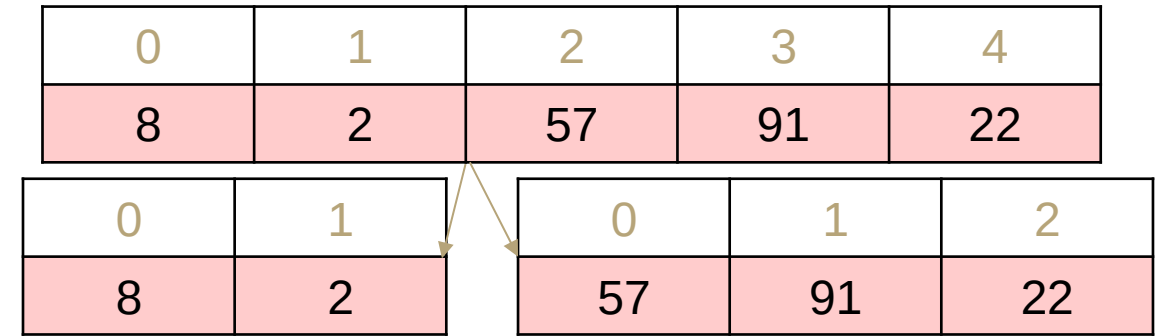
Mergesort

0	1	2	3	4
8	2	57	91	22

```
mergeSort(list) {
    if (list.length <= 1)
        return list
    else
        leftHalf = mergeSort(list[0, mid])
        rightHalf = mergeSort(list[mid + 1, ...])
        return merge(leftHalf, rightHalf)
}
```

Mergesort

Divide



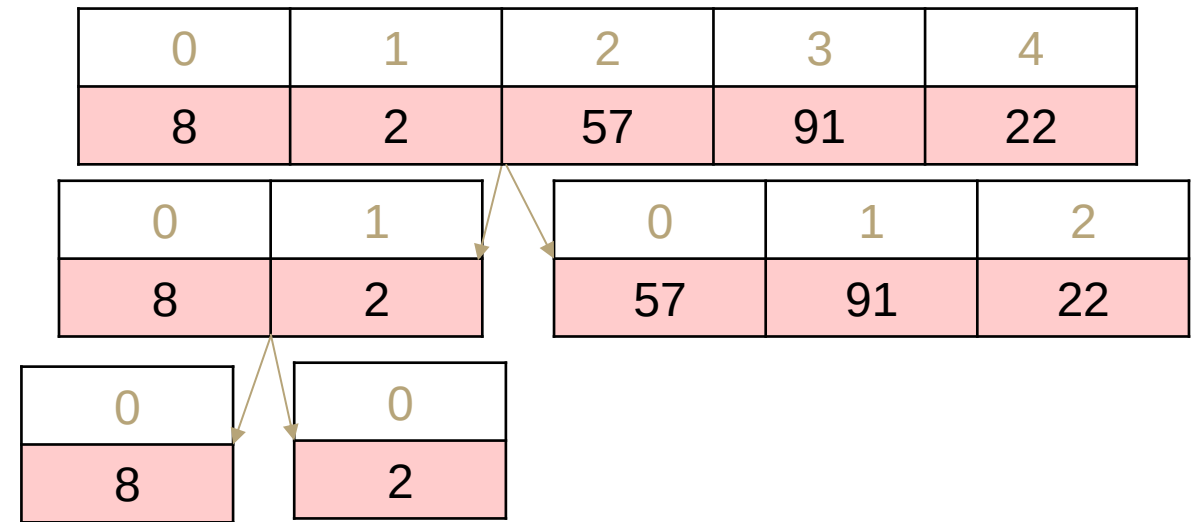
```
mergeSort(list) {
  if (list.length <= 1)
    return list
  else
    leftHalf = mergeSort(list[0, mid])
    rightHalf = mergeSort(list[mid + 1, ...])
    return merge(leftHalf, rightHalf)
}
```

Mergesort

Divide

Conquer

```
mergeSort(list) {
  if (list.length <= 1)
    return list
  else
    leftHalf = mergeSort(list[0, mid])
    rightHalf = mergeSort(list[mid + 1, ...])
    return merge(leftHalf, rightHalf)
}
```

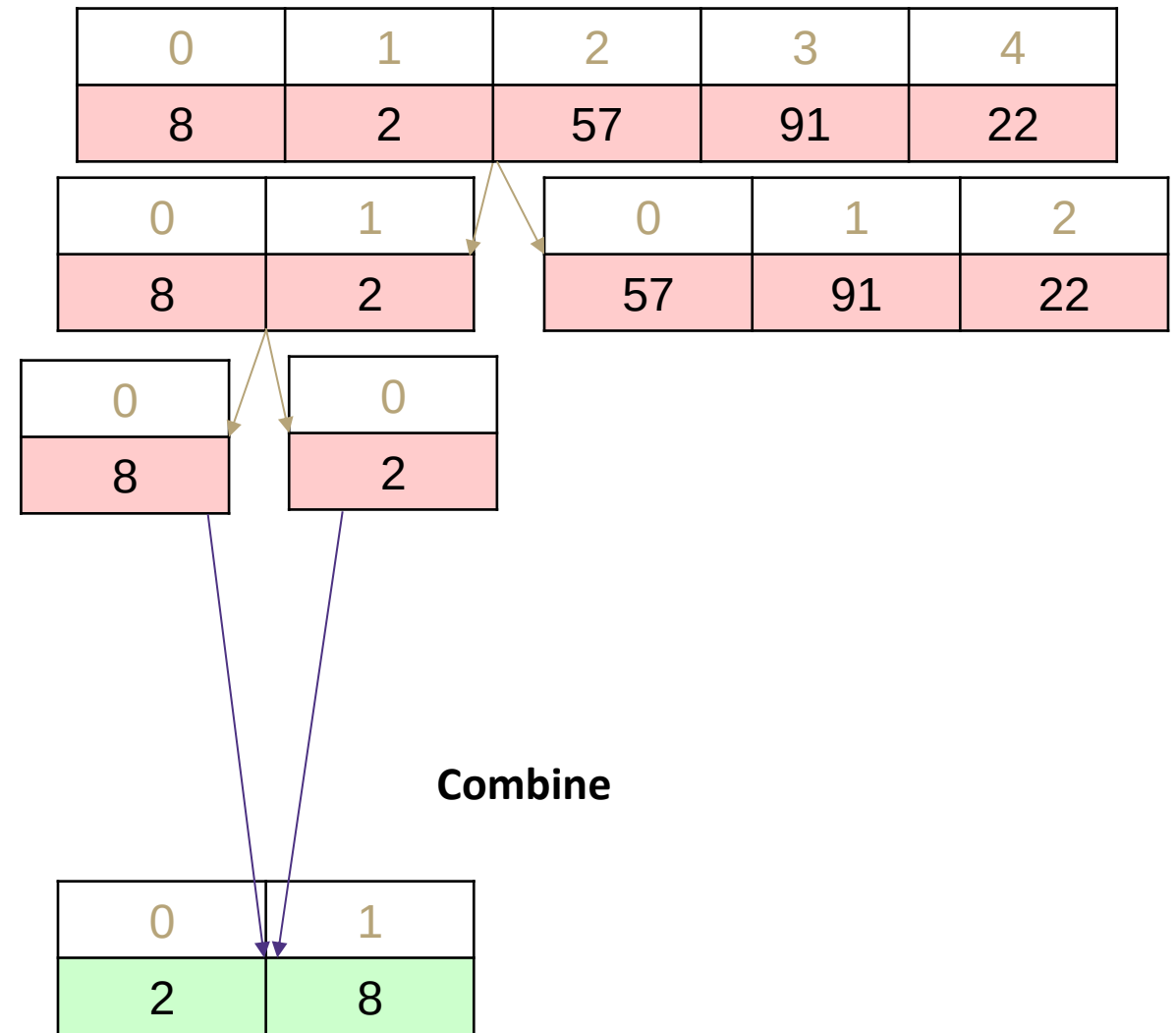


Mergesort

```
mergeSort(list) {
  if (list.length <= 1)
    return list
  else
    leftHalf = mergeSort(list[0, mid])
    rightHalf = mergeSort(list[mid + 1, ...])
    return merge(leftHalf, rightHalf)
}
```

Divide

Conquer

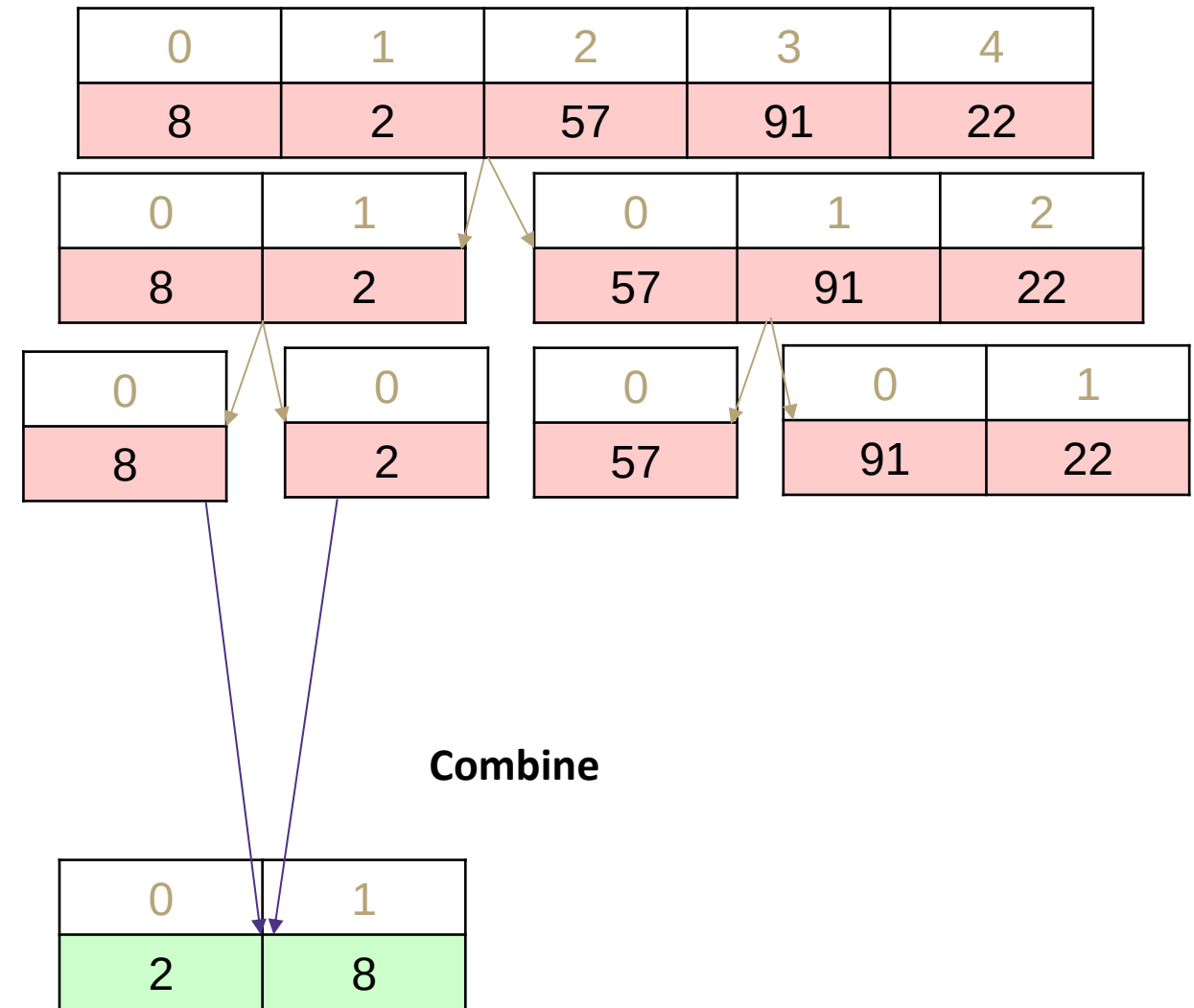


Mergesort

```
mergeSort(list) {
  if (list.length <= 1)
    return list
  else
    leftHalf = mergeSort(list[0, mid])
    rightHalf = mergeSort(list[mid + 1, ...])
    return merge(leftHalf, rightHalf)
}
```

Divide

Conquer



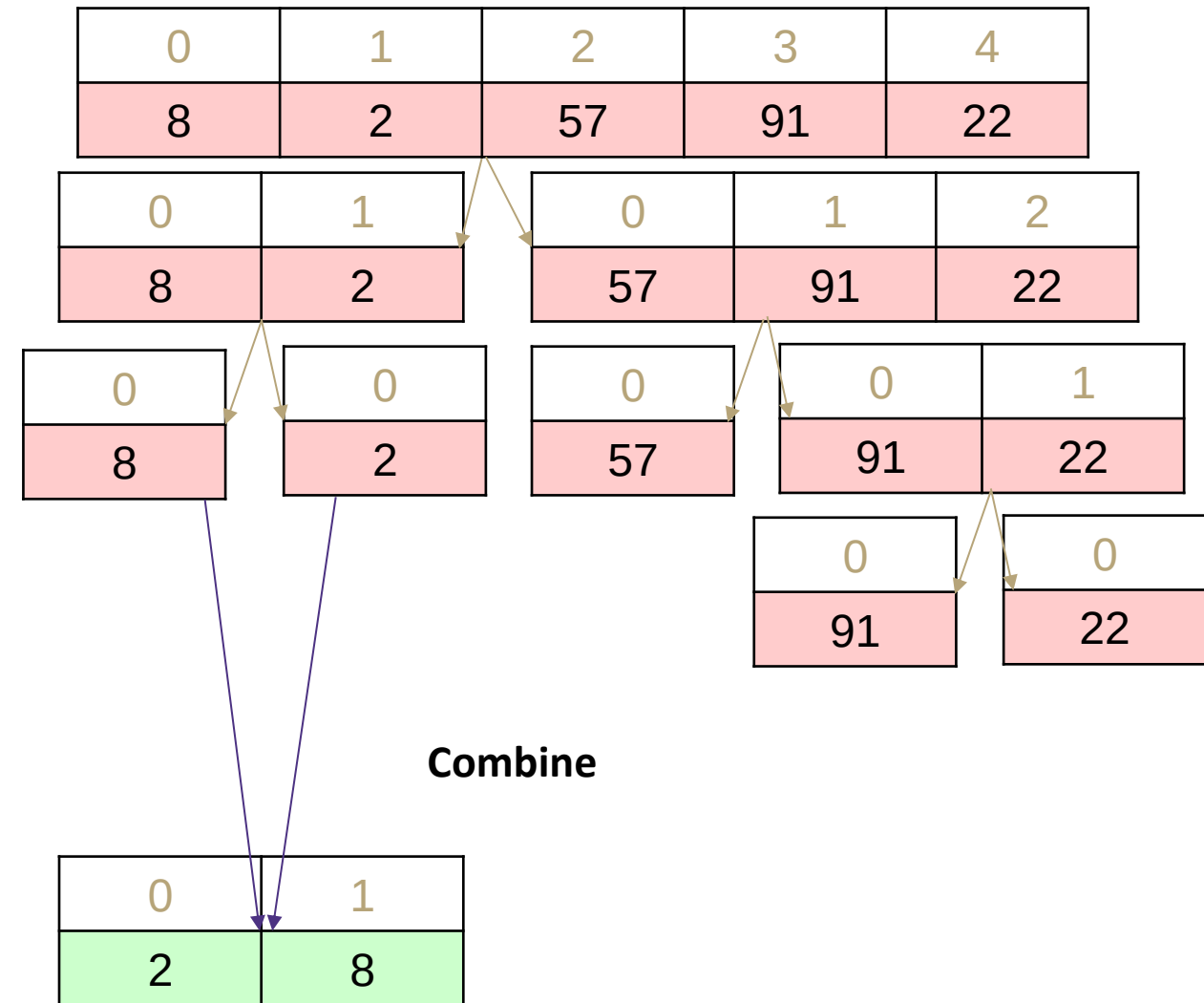
Combine

Mergesort

```
mergeSort(list) {
  if (list.length <= 1)
    return list
  else
    leftHalf = mergeSort(list[0, mid])
    rightHalf = mergeSort(list[mid + 1, ...])
    return merge(leftHalf, rightHalf)
}
```

Divide

Conquer



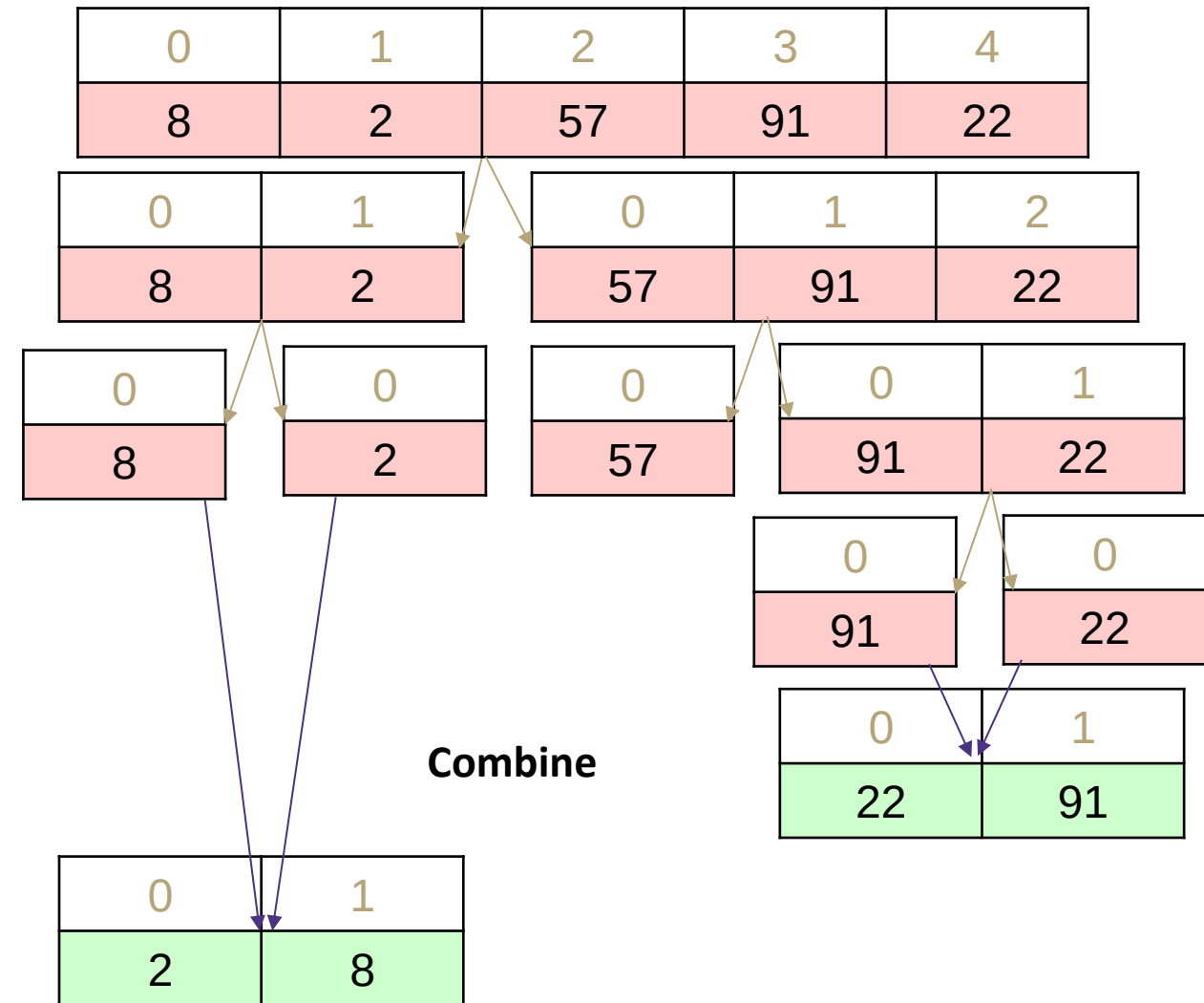
Combine

Mergesort

```
mergeSort(list) {
  if (list.length <= 1)
    return list
  else
    leftHalf = mergeSort(list[0, mid])
    rightHalf = mergeSort(list[mid + 1, ...])
    return merge(leftHalf, rightHalf)
}
```

Divide

Conquer

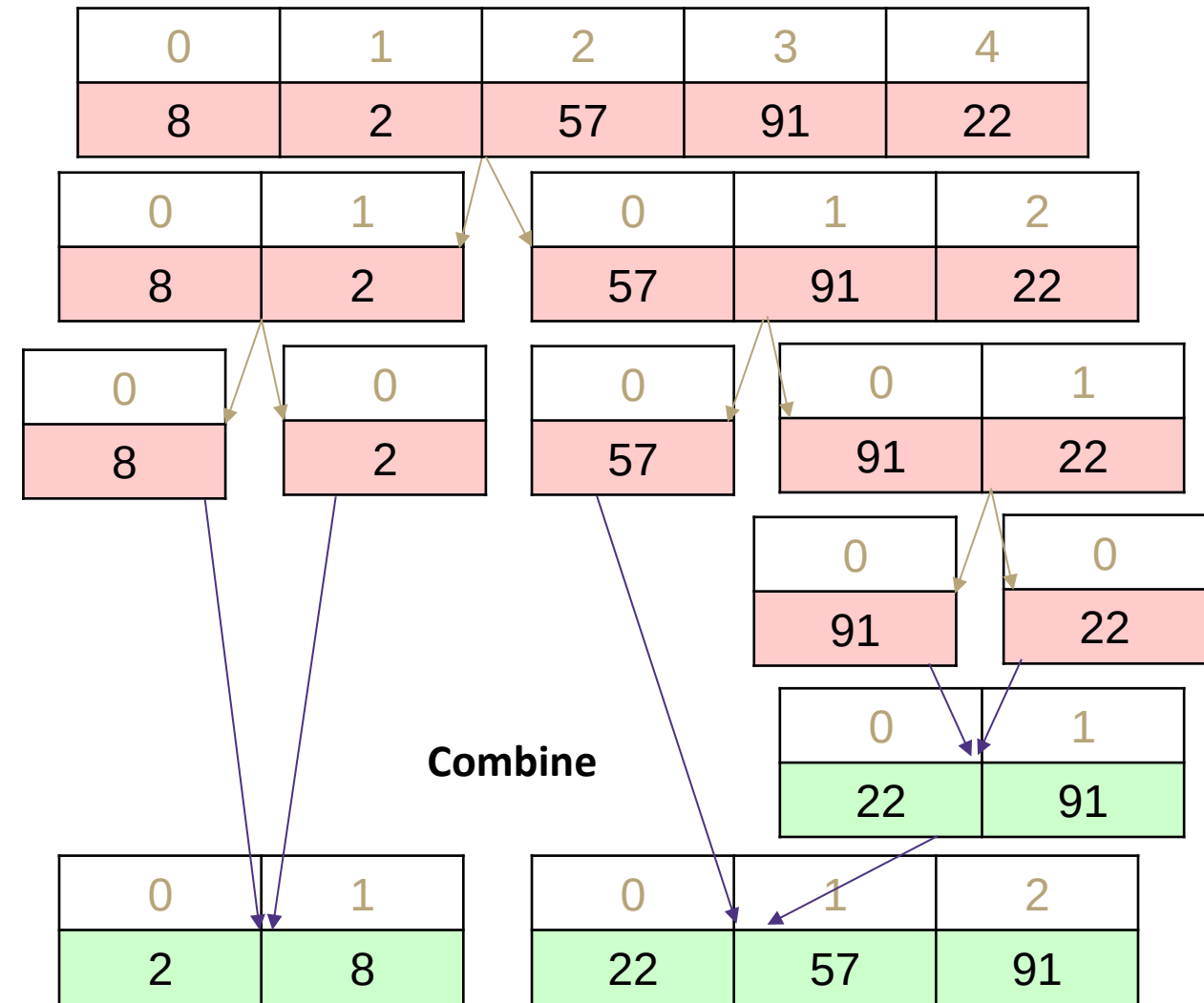


Mergesort

```
mergeSort(list) {
  if (list.length <= 1)
    return list
  else
    leftHalf = mergeSort(list[0, mid])
    rightHalf = mergeSort(list[mid + 1, ...])
    return merge(leftHalf, rightHalf)
}
```

Divide

Conquer

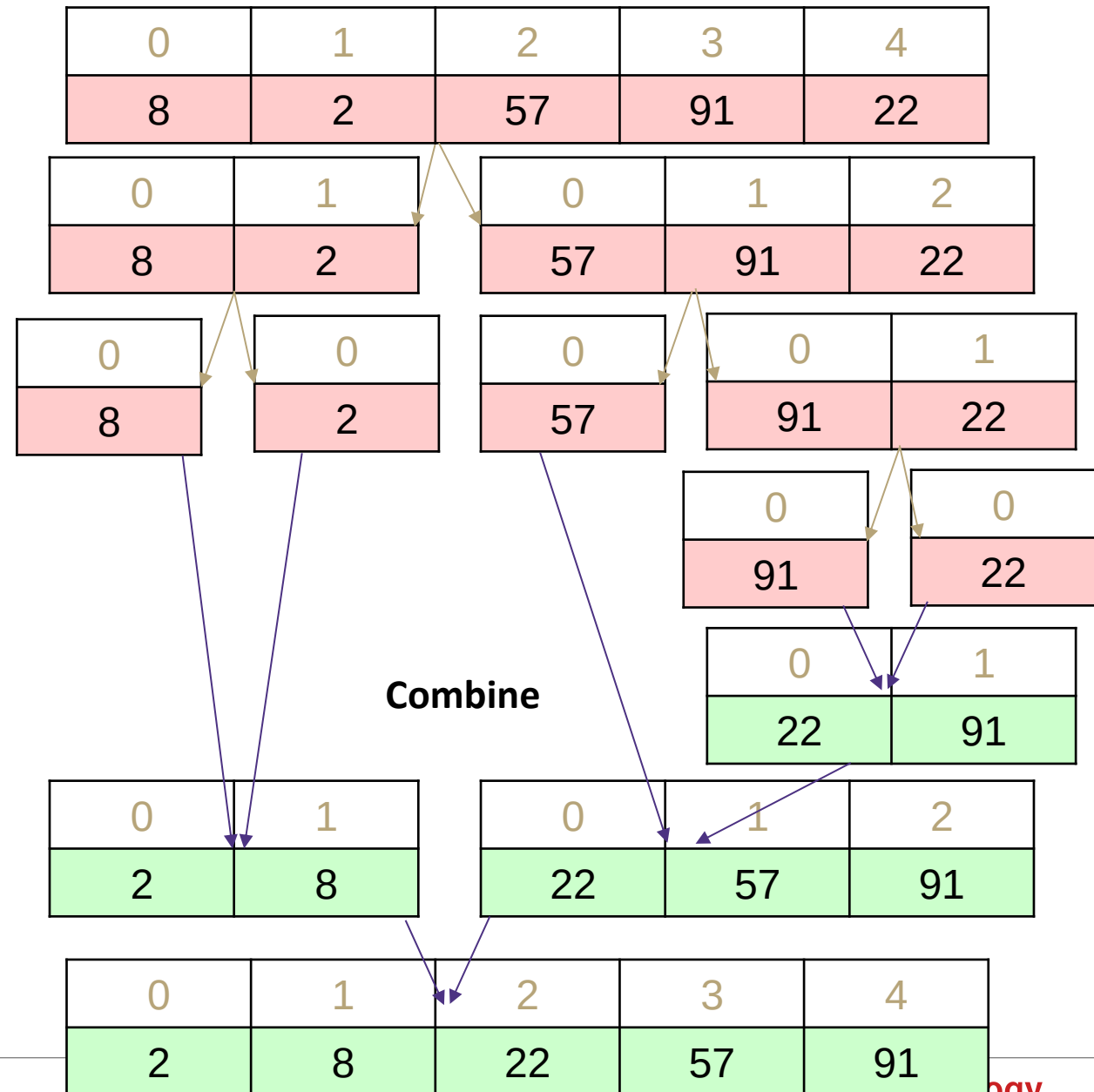


Mergesort

```
mergeSort(list) {
  if (list.length <= 1)
    return list
  else
    leftHalf = mergeSort(list[0, mid])
    rightHalf = mergeSort(list[mid + 1, ...])
    return merge(leftHalf, rightHalf)
}
```

Divide

Conquer



Mergesort

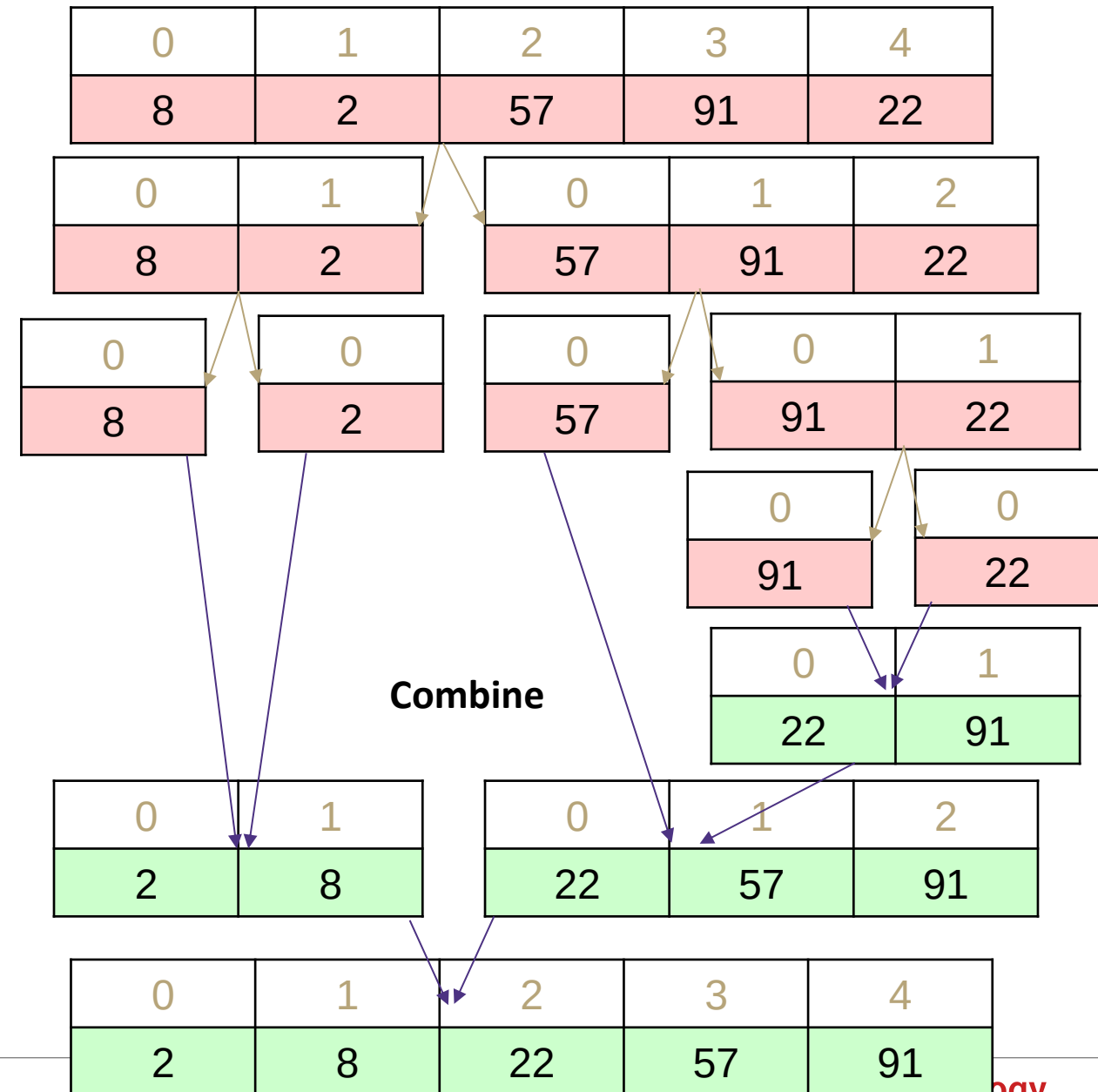
```
mergeSort(list) {
  if (list.length <= 1)
    return list
  else
    leftHalf = mergeSort(list[0, mid])
    rightHalf = mergeSort(list[mid + 1, ...])
    return merge(leftHalf, rightHalf)
}
```

Muster #2 – Teile-und-Beherrsche

Divide

Conquer

Combine



Mergesort

```
mergeSort(list) {
  if (list.length <= 1)
    return list
  else
    leftHalf = mergeSort(list[0, mid])
    rightHalf = mergeSort(list[mid + 1, ...])
    return merge(leftHalf, rightHalf)
}
```

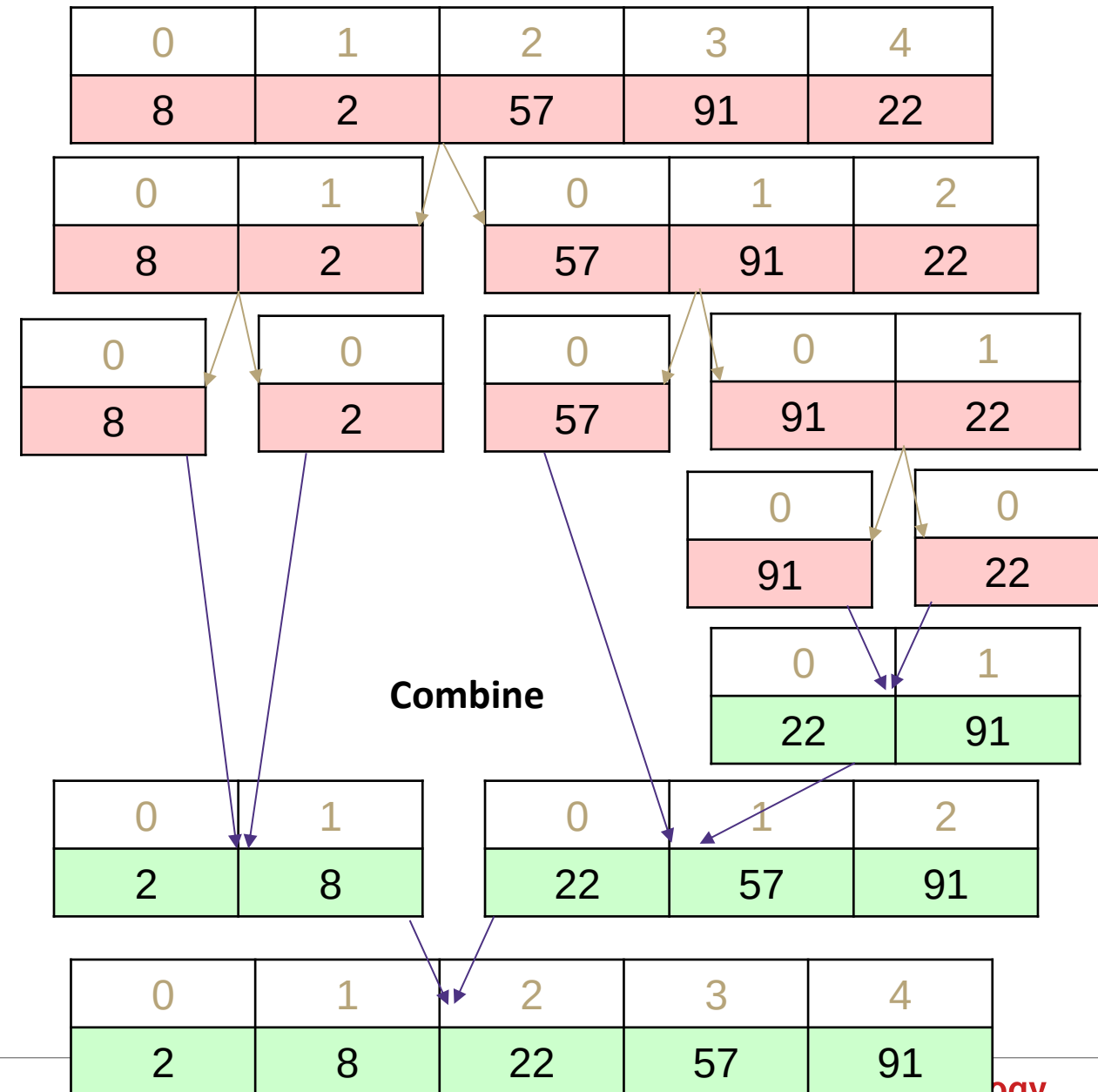
$$T(n) = \begin{cases} 2 & \text{falls } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{sonst} \end{cases}$$

Muster #2 – Teile-und-Beherrsche

Divide

Conquer

Combine



Mergesort Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 2 & \text{falls } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

Mergesort Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 2 & \text{falls } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$a = 2, b = 2, c = 1$$

Mergesort Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 2 & \text{falls } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$a = 2, b = 2, c = 1$$

$$x = \log_b y \text{ entspricht } b^x = y$$

Mergesort Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 2 & \text{falls } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$a = 2, b = 2, c = 1$$

$$x = \log_b y \text{ entspricht } b^x = y$$

$$\log_2 2 = x \implies 2^x = 2 \implies x = 1$$

Mergesort Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 2 & \text{falls } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$a = 2, b = 2, c = 1$$

$$x = \log_b y \text{ entspricht } b^x = y$$

$$\log_2 2 = x \implies 2^x = 2 \implies x = 1$$

$$\log_2 2 = 1$$

Mergesort Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} 2 & \text{falls } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Es gilt $f(n) \in \Theta(n^c)$

Wenn $\log_b a < c$, dann $T(n) \in \Theta(n^c)$

Wenn $\log_b a = c$, dann $T(n) \in \Theta(n^c \log n)$

Wenn $\log_b a > c$, dann $T(n) \in \Theta(n^{\log_b a})$

$$a = 2, b = 2, c = 1$$

$$x = \log_b y \text{ entspricht } b^x = y$$

$$\log_2 2 = x \implies 2^x = 2 \implies x = 1$$

$$\log_2 2 = 1$$

Wir sind in Fall 2: $T(n) \in \Theta(n \log n)$

Teile-und-Beherrsche

```
procedure p(input x of size n):
```

```
    if n < some constant k:
```

```
        Solve x directly without recursion
```

```
    else:
```

```
        Create a subproblems of x, each having size n/b (Divide)
```

```
        Call procedure p recursively on each subproblem (Conquer)
```

```
        Combine the results from the subproblems (Combine)
```

```
mergeSort(list) {  
    if (list.length <= 1)  
        return list  
    else  
        leftHalf = mergeSort(list[0, mid])  
        rightHalf = mergeSort(list[mid + 1, ...])  
        return merge(leftHalf, rightHalf)  
}
```

Binäre Suche als Teile-und-Beherrsche

```
procedure p(input x of size n):  
    if n < some constant k:  
        Solve x directly without recursion  
    else:  
        Create 1 subproblem of x, each having size n/2  
        Call procedure p recursively on each subproblem  
        Combine the results from the subproblems
```

Fragen?

- Mergesort

Code Modellierung: Rekursive Muster

Muster Nr. 1: Halbierung der Eingabe

- Binäre Suche – Laufzeit $\Theta(\log n)$

Muster Nr. 2: Teilen der Eingabe, Beherrschen und Vereinigen (Teile-und-Beherrsche)

- Mergesort – Laufzeit $\Theta(n \log n)$

Muster Nr. 3: Verdoppelung der Eingabe

- Berechnung der Fibonacci-Zahlen

Bsp. für Muster 3: Berechnung der Fibonacci-Zahlen

0, 1, 1, 2, 3, 5, 8, 13, 21

Die n te Fibonacci Zahl ist die Summe aus der $n-1$ ten und der $n-2$ ten Fibonacci Zahl.

Per Definition ist:

- die 0te Fibonacci-Zahl: 0
- die 1te Fibonacci-Zahl: 1

Bsp. für Muster 3: Berechnung der Fibonacci-Zahlen

0, 1, 1, 2, 3, 5, 8, 13, 21

```
public int fib(int n) {  
    if (n <= 1) {  
        return n;  
    }  
    return fib(n-1) + fib(n-2);  
}
```

Die nte Fibonacci Zahl ist die Summe aus der n-1ten und der n-2ten Fibonacci Zahl.

Per Definition ist:

- die 0te Fibonacci-Zahl: 0
- die 1te Fibonacci-Zahl: 1

Bsp. für Muster 3: Berechnung der Fibonacci-Zahlen

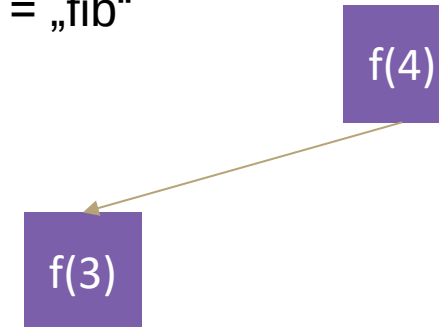
0, 1, 1, 2, 3, 5, 8, 13, 21

```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

-
-
-

Funktionsaufrufe für n=4 (Rekursionsbaum):

„f“ = „fib“



Bsp. für Muster 3: Berechnung der Fibonacci-Zahlen

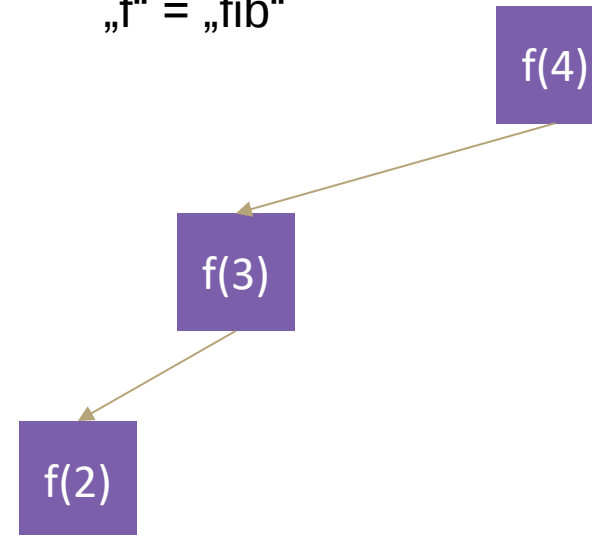
0, 1, 1, 2, 3, 5, 8, 13, 21

```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

-
-
-

Funktionsaufrufe für n=4 (Rekursionsbaum):

„f“ = „fib“



Bsp. für Muster 3: Berechnung der Fibonacci-Zahlen

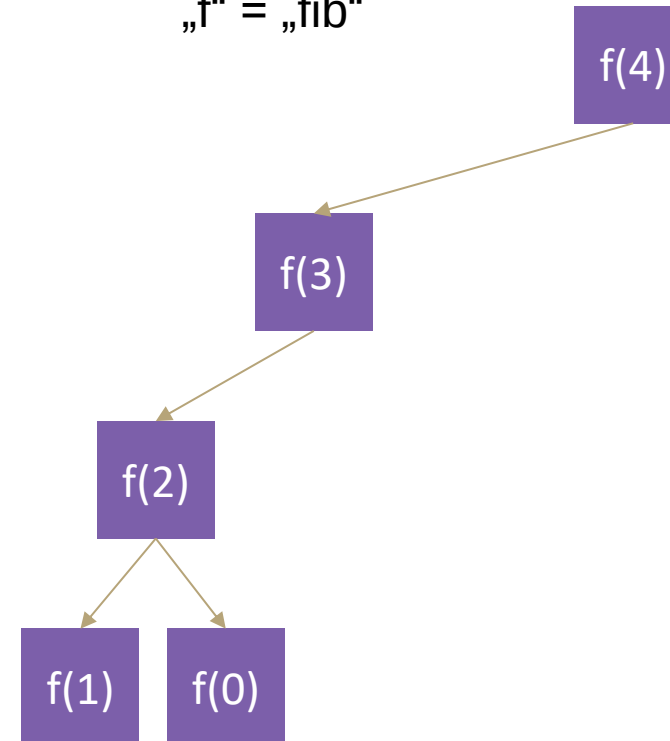
0, 1, 1, 2, 3, 5, 8, 13, 21

```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

-
-
-

Funktionsaufrufe für n=4 (Rekursionsbaum):

„f“ = „fib“



Bsp. für Muster 3: Berechnung der Fibonacci-Zahlen

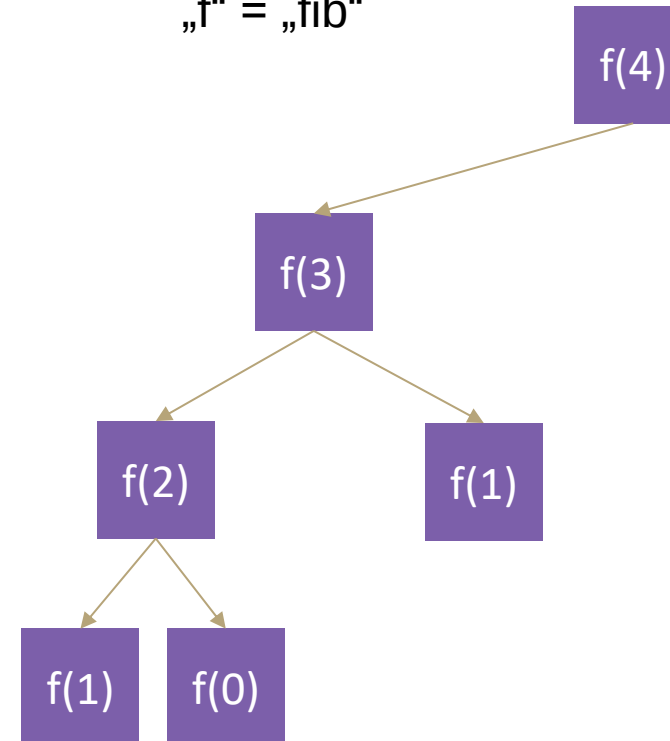
0, 1, 1, 2, 3, 5, 8, 13, 21

```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

-
-
-

Funktionsaufrufe für n=4 (Rekursionsbaum):

„f“ = „fib“



Bsp. für Muster 3: Berechnung der Fibonacci-Zahlen

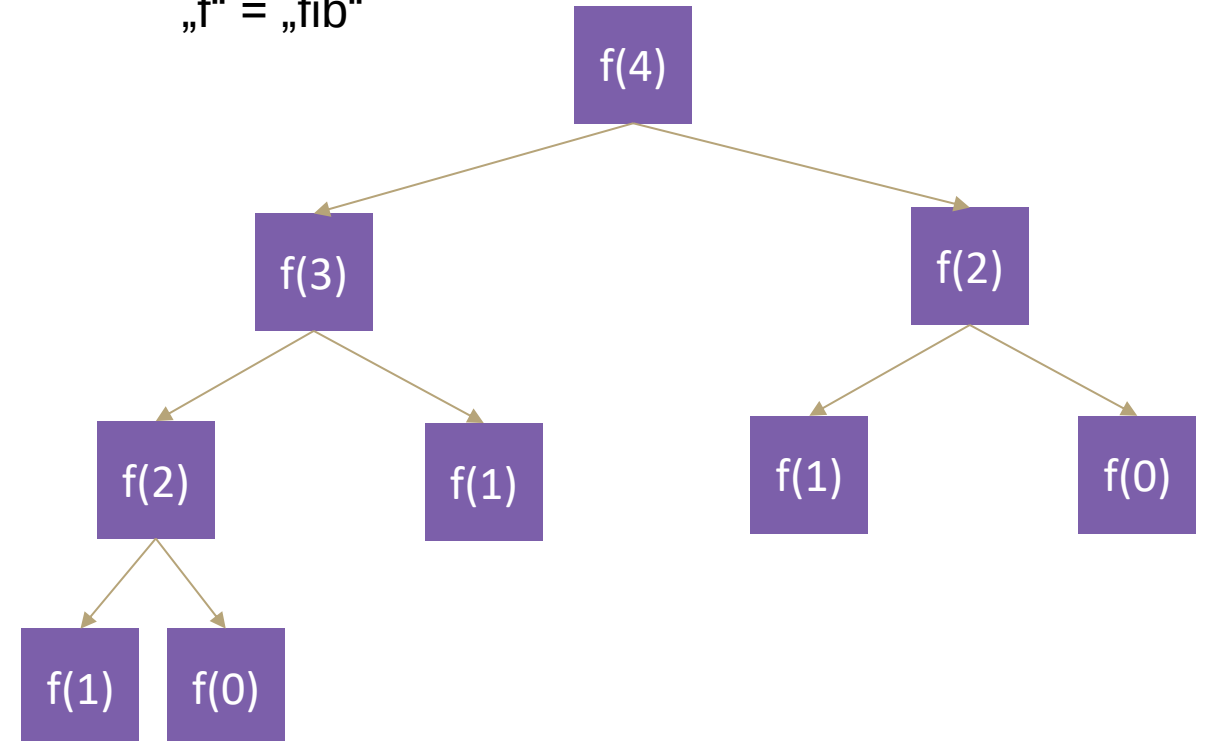
0, 1, 1, 2, 3, 5, 8, 13, 21

```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

-
-
-

Funktionsaufrufe für n=4 (Rekursionsbaum):

„f“ = „fib“



Bsp. für Muster 3: Berechnung der Fibonacci-Zahlen

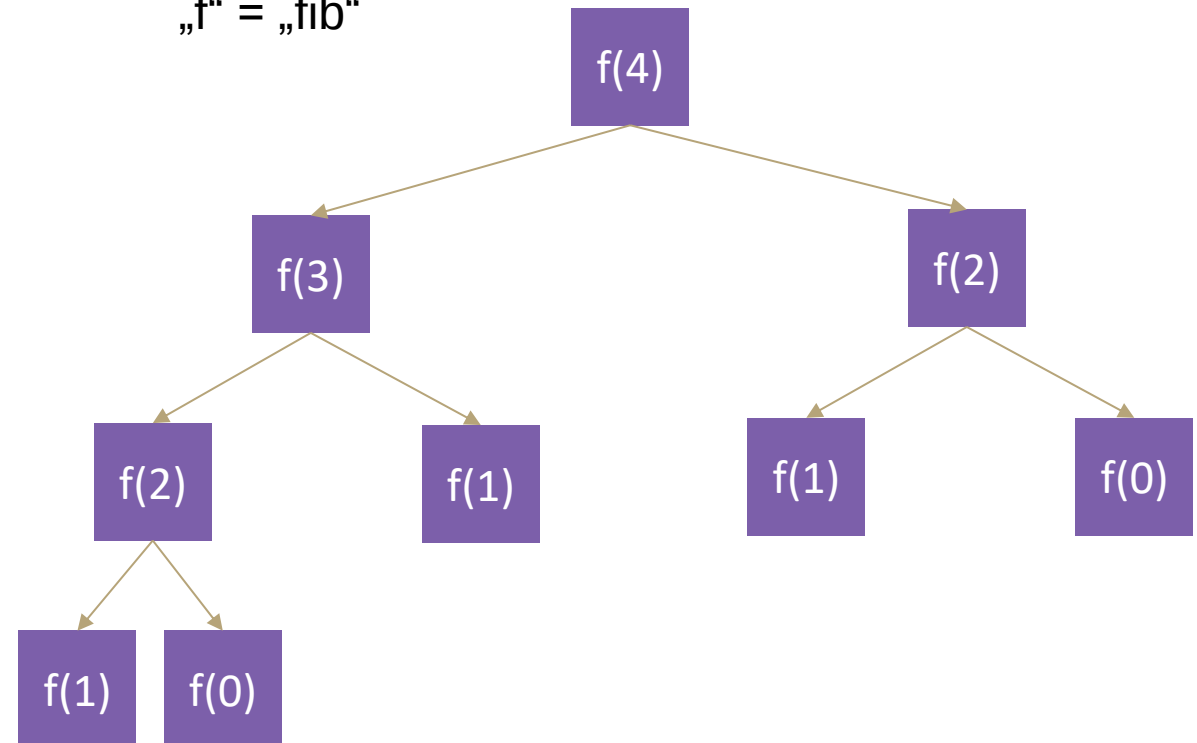
0, 1, 1, 2, 3, 5, 8, 13, 21

```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

- Jeder Aufruf erzeugt 2 weitere Aufrufe
- Jeder neue Aufruf hat eine Kopie der Eingabe (fast)
- Fast Verdoppelung der Eingabe bei jedem Aufruf

Funktionsaufrufe für n=4 (Rekursionsbaum):

„f“ = „fib“



Bsp. für Muster 3: Berechnung der Fibonacci-Zahlen

0, 1, 1, 2, 3, 5, 8, 13, 21

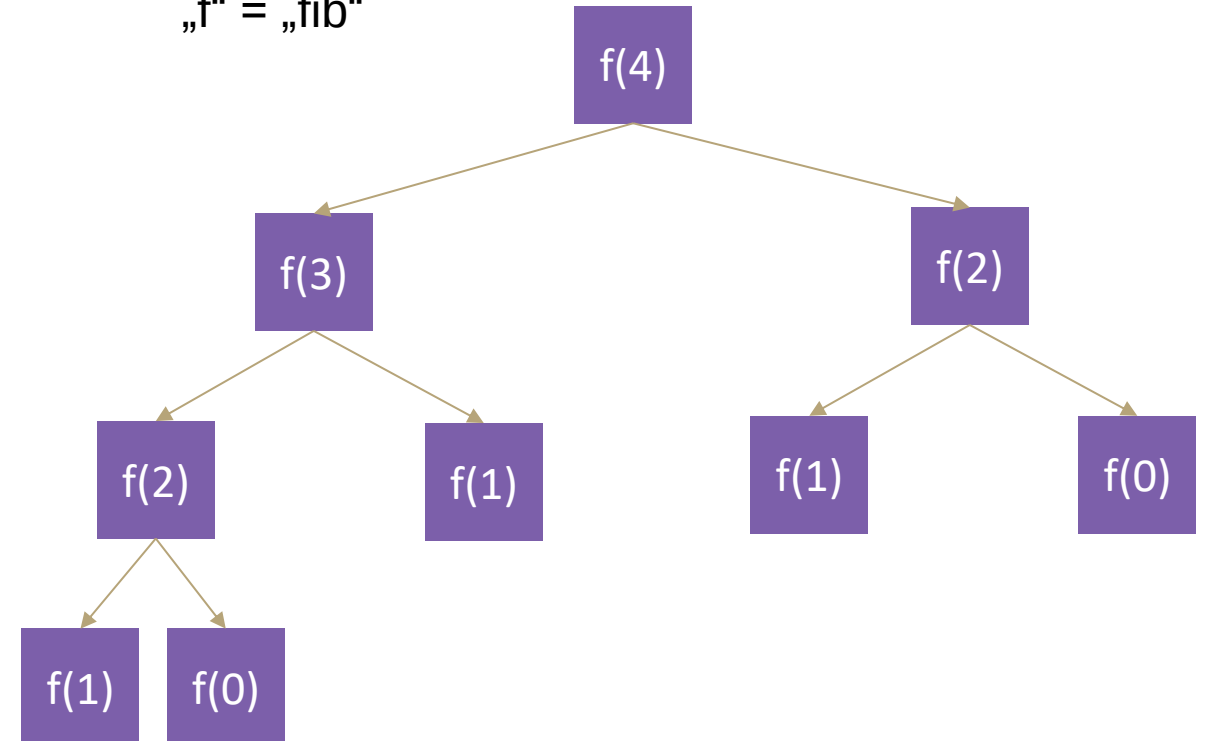
```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

- Jeder Aufruf erzeugt 2 weitere Aufrufe
- Jeder neue Aufruf hat eine Kopie der Eingabe (fast)
- Fast Verdoppelung der Eingabe bei jedem Aufruf

Muster #3 – Verdoppelung der Eingabe

Funktionsaufrufe für n=4 (Rekursionsbaum):

„f“ = „fib“



Bsp. für Muster 3: Berechnung der Fibonacci-Zahlen

0, 1, 1, 2, 3, 5, 8, 13, 21

```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

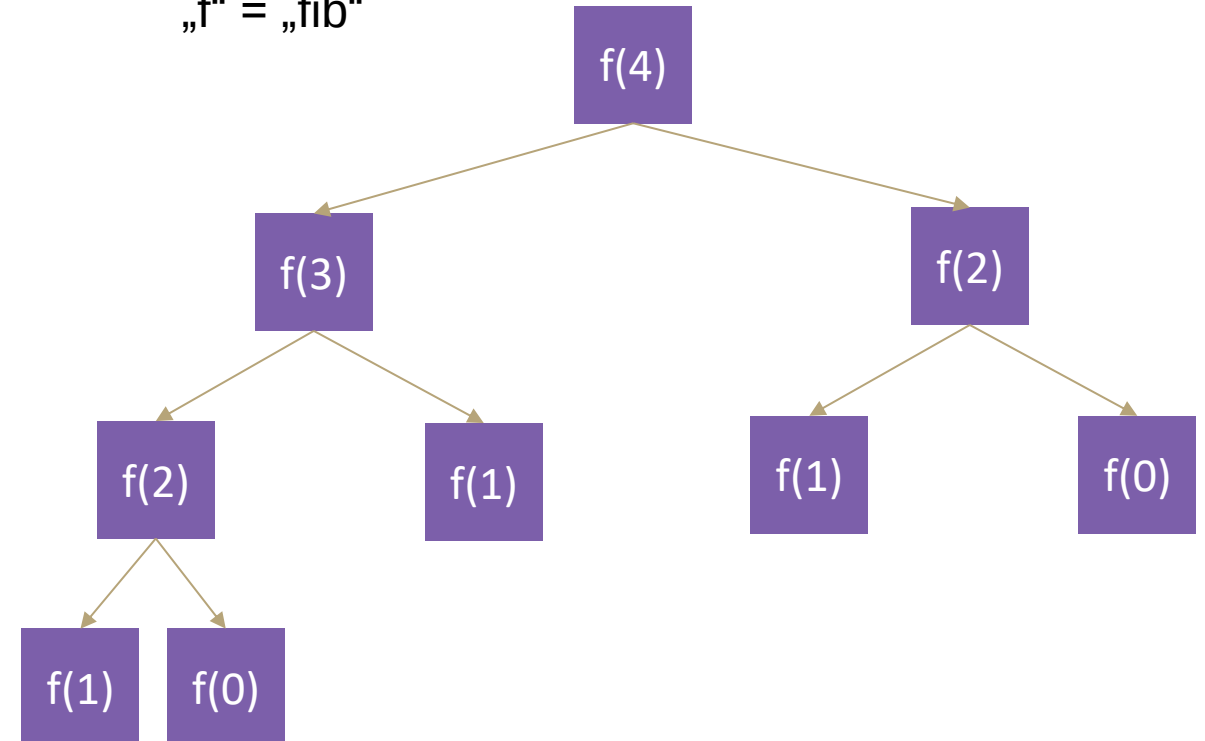
- Jeder Aufruf erzeugt 2 weitere Aufrufe
- Jeder neue Aufruf hat eine Kopie der Eingabe (fast)
- Fast Verdoppelung der Eingabe bei jedem Aufruf

Muster #3 – Verdoppelung der Eingabe

Fast

Funktionsaufrufe für n=4 (Rekursionsbaum):

„f“ = „fib“



Berechnung der Fibonacci-Zahlen

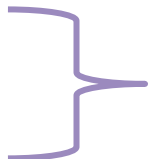
Rekursionsgleichung in Θ -Notation umwandeln

```
public int fib(int n) {  
    if (n <= 1) {  
        return n;  
    }  
    return fib(n-1) + fib(n-2);  
}
```

Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

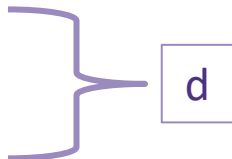
```
public int fib(int n) {  
    if (n <= 1) {  
        return n;  
    }  
    return fib(n-1) + fib(n-2);  
}
```



Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

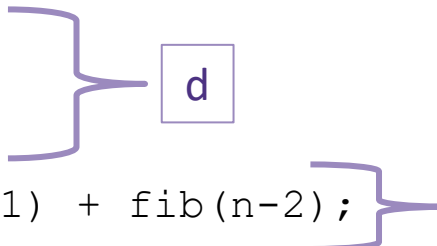
```
public int fib(int n) {  
    if (n <= 1) {  
        return n;  
    }  
    return fib(n-1) + fib(n-2);  
}
```



Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

```
public int fib(int n) {  
    if (n <= 1) {  
        return n;  
    }  
    return fib(n-1) + fib(n-2);  
}
```



Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

Diagram illustrating the time complexity analysis of the Fibonacci function:

- The base case `if (n <= 1) { return n; }` is annotated with a bracket and a box labeled **d**, representing a constant time cost.
- The recursive case `return fib(n-1) + fib(n-2);` is annotated with a bracket and a box labeled **$T(n-1) + T(n-2) + 4$** , representing the time cost of the recursive calls and the addition operation.

Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

Diagramm zur Rekursionsgleichung:

- Der Codeblock `if (n <= 1) { return n; }` ist mit einer Klammer und einem Kasten mit der Beschriftung **d** markiert.
- Der Codeblock `return fib(n-1) + fib(n-2);` ist mit einer Klammer und einem Kasten mit der Beschriftung **T(n-1) + T(n-2) + 4** markiert.

$$T(n) = \begin{cases} d & \text{falls } n \leq 1 \\ T(n-1) + T(n-2) + 5 & \text{sonst} \end{cases}$$

Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

Diagramm zur Rekursionsgleichung:

- Die Basisfall-Logik (`if (n <= 1) { return n; }`) ist mit einer Klammer und einem Kasten mit der Beschriftung **d** markiert.
- Die rekursive Aufruf-Logik (`return fib(n-1) + fib(n-2);`) ist mit einer Klammer und einem Kasten mit der Beschriftung **$T(n-1) + T(n-2) + 4$** markiert.

Können wir die Mastermethode verwenden?

$$T(n) = \begin{cases} d & \text{falls } n \leq 1 \\ T(n-1) + T(n-2) + 5 & \text{sonst} \end{cases}$$

Mastermethode

$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

Diagramm zur Rekursionsgleichung:

- Die Basisfall-Condition `if (n <= 1) { return n; }` ist mit einer Klammer und einem Kasten mit der Beschriftung **d** markiert.
- Der rekursive Aufruf `return fib(n-1) + fib(n-2);` ist mit einer Klammer und einem Kasten mit der Beschriftung **$T(n-1) + T(n-2) + 4$** markiert.

Können wir die Mastermethode verwenden?

Nein, unsere Rekursionsgleichung passt nicht in dieses Format...

$$T(n) = \begin{cases} d & \text{falls } n \leq 1 \\ T(n-1) + T(n-2) + 5 & \text{sonst} \end{cases}$$

Mastermethode

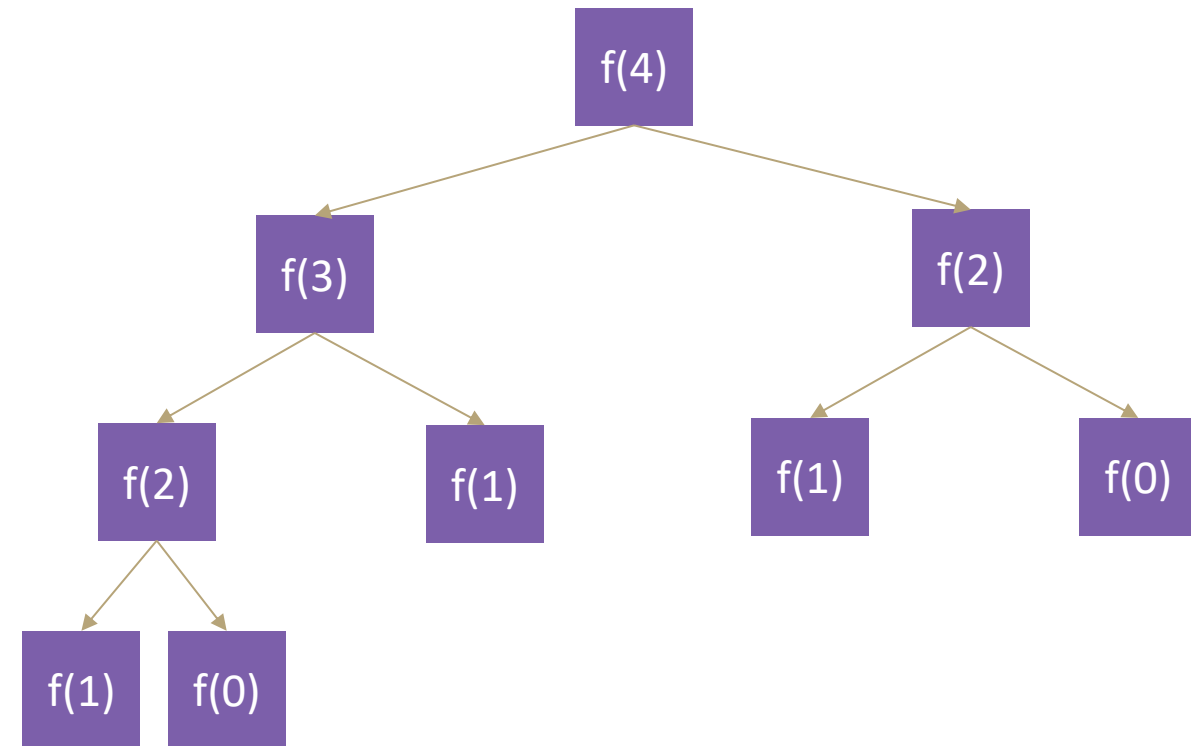
$$T(n) = \begin{cases} d & \text{falls } n \text{ höchstens eine Konstante ist} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{sonst} \end{cases}$$

Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} d & \text{falls } n \leq 1 \\ T(n-1) + T(n-2) + 7 & \text{sonst} \end{cases}$$

Wie viele Ebenen enthält der Rekursionsbaum?



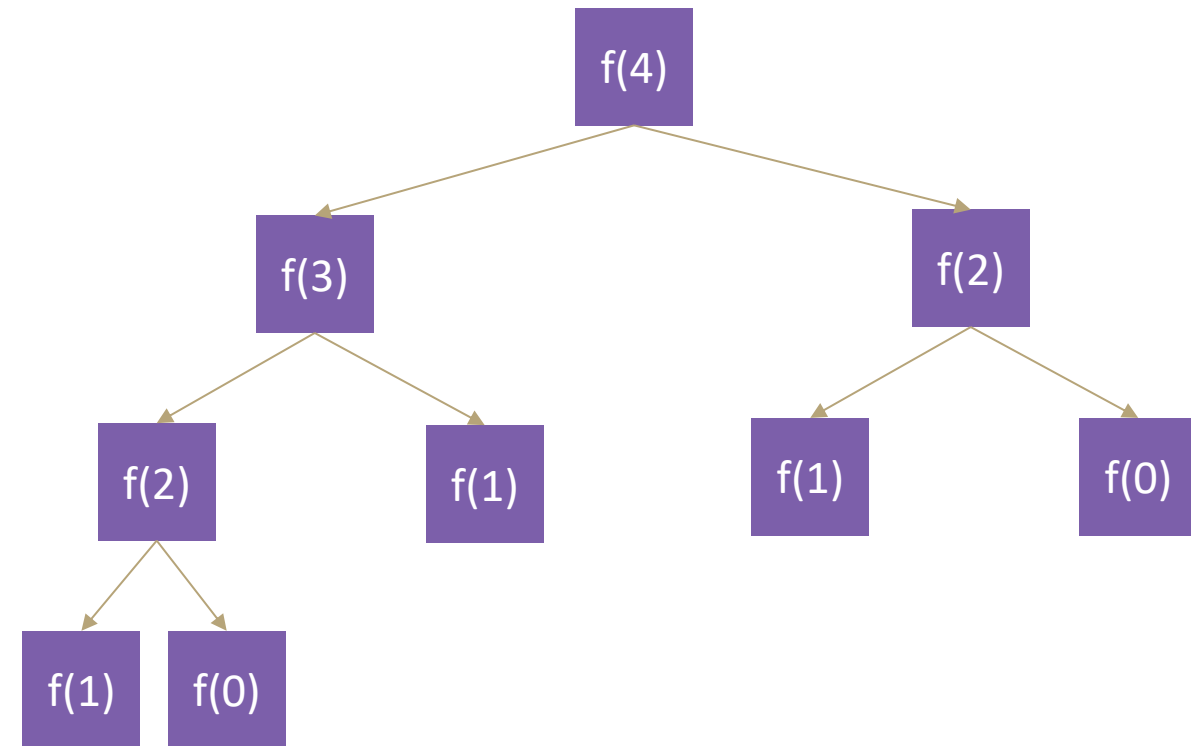
Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} d & \text{falls } n \leq 1 \\ T(n-1) + T(n-2) + 7 & \text{sonst} \end{cases}$$

Wie viele Ebenen enthält der Rekursionsbaum?

Wie viele Ebenen sind nötig, um „n“ durch Subtraktion von 1 in den Basisfall „1“ umzuwandeln?



Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

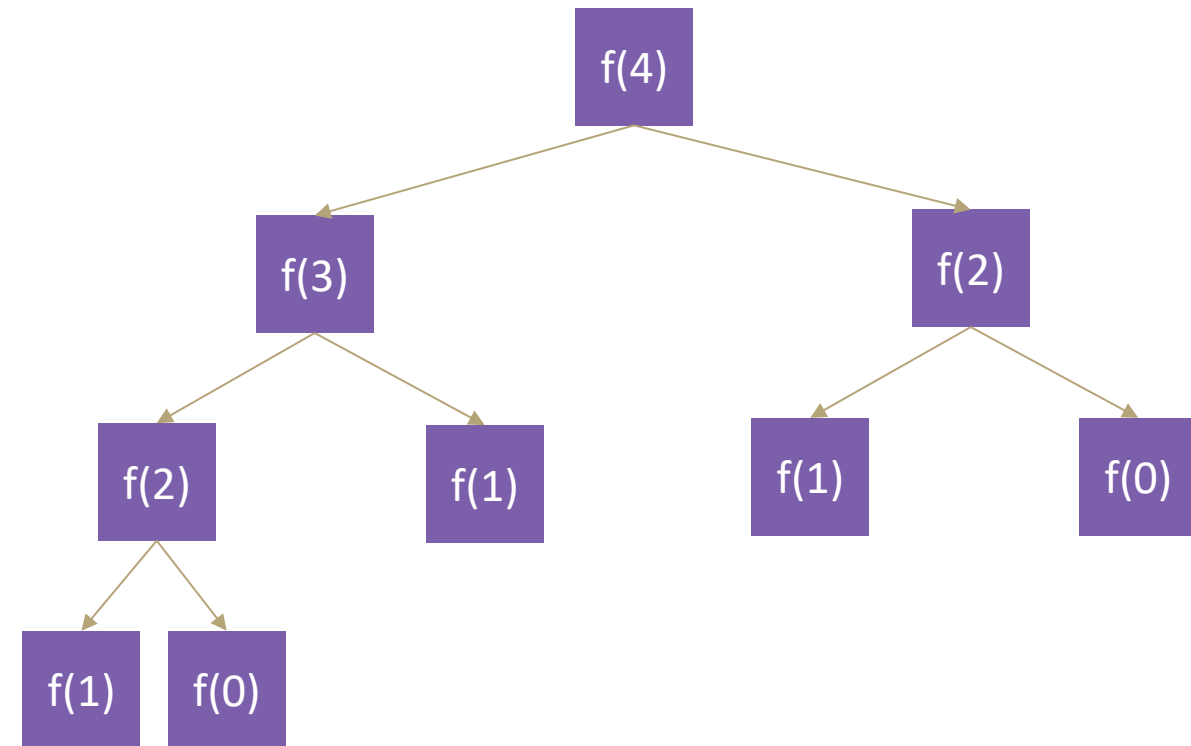
$$T(n) = \begin{cases} d & \text{falls } n \leq 1 \\ T(n-1) + T(n-2) + 7 & \text{sonst} \end{cases}$$

Wie viele Ebenen enthält der Rekursionsbaum?

Wie viele Ebenen sind nötig, um „n“ durch Subtraktion von 1 in den Basisfall „1“ umzuwandeln?

Für unser Beispiel: 4 -> Höhe = n

Wie viele Funktionsaufrufe pro Ebene?



Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} d & \text{falls } n \leq 1 \\ T(n-1) + T(n-2) + 7 & \text{sonst} \end{cases}$$

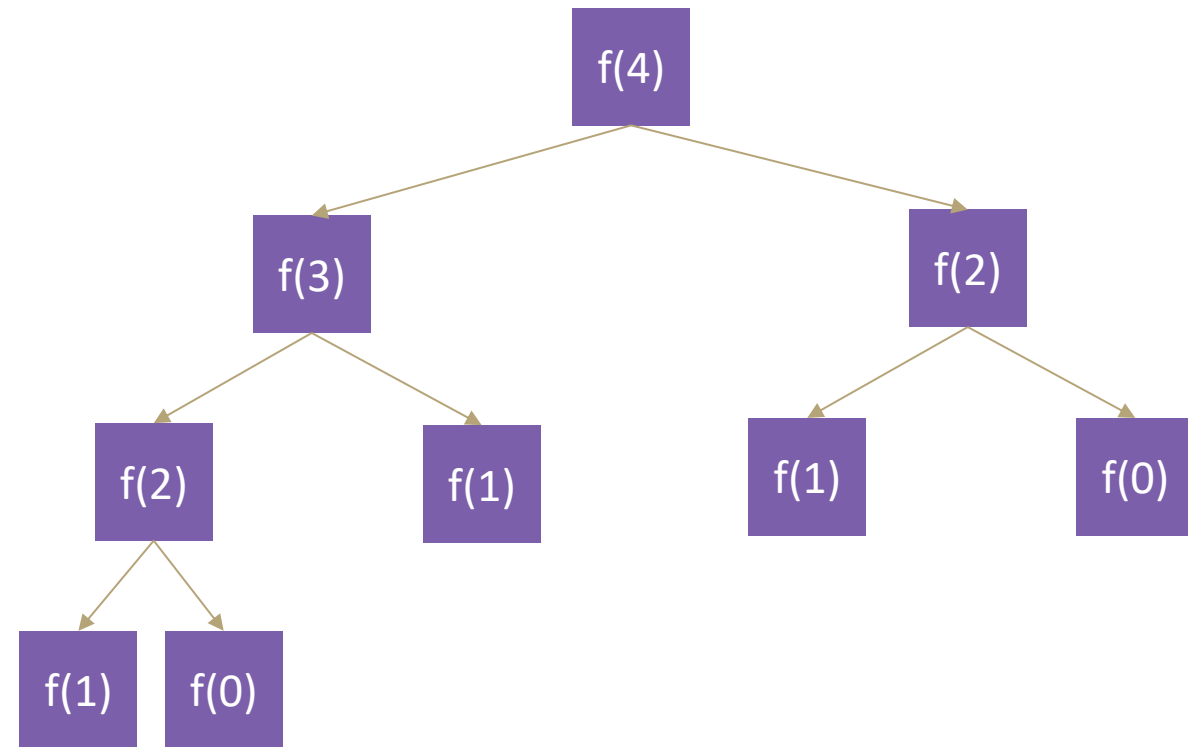
Wie viele Ebenen enthält der Rekursionsbaum?

Wie viele Ebenen sind nötig, um „n“ durch Subtraktion von 1 in den Basisfall „1“ umzuwandeln?

Für unser Beispiel: 4 -> Höhe = n

Wie viele Funktionsaufrufe pro Ebene?

Ebene	Funktionsaufrufe
1	1
2	2
3	4
4	8



Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} d & \text{falls } n \leq 1 \\ T(n-1) + T(n-2) + 7 & \text{sonst} \end{cases}$$

Wie viele Ebenen enthält der Rekursionsbaum?

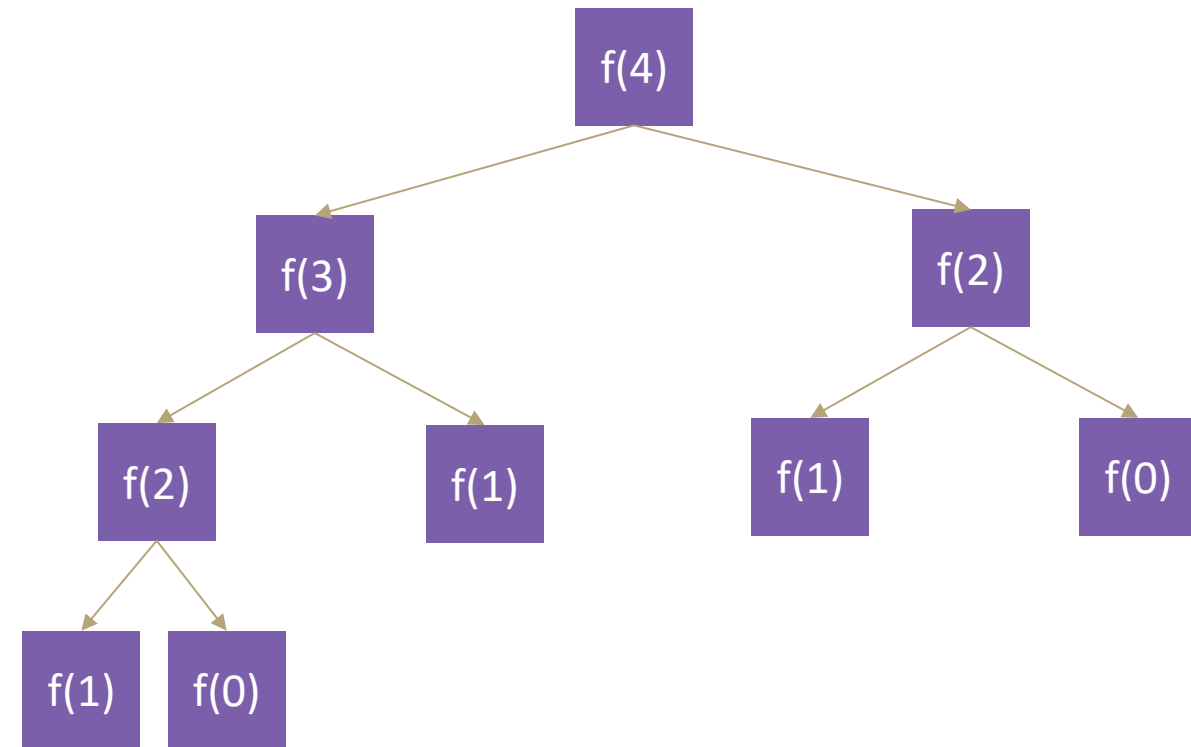
Wie viele Ebenen sind nötig, um „n“ durch Subtraktion von 1 in den Basisfall „1“ umzuwandeln?

Für unser Beispiel: 4 -> Höhe = n

Wie viele Funktionsaufrufe pro Ebene?

Wie viele Funktionsaufrufe auf Ebene k?

Ebene	Funktionsaufrufe
1	1
2	2
3	4
4	8



Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} d & \text{falls } n \leq 1 \\ T(n-1) + T(n-2) + 7 & \text{sonst} \end{cases}$$

Wie viele Ebenen enthält der Rekursionsbaum?

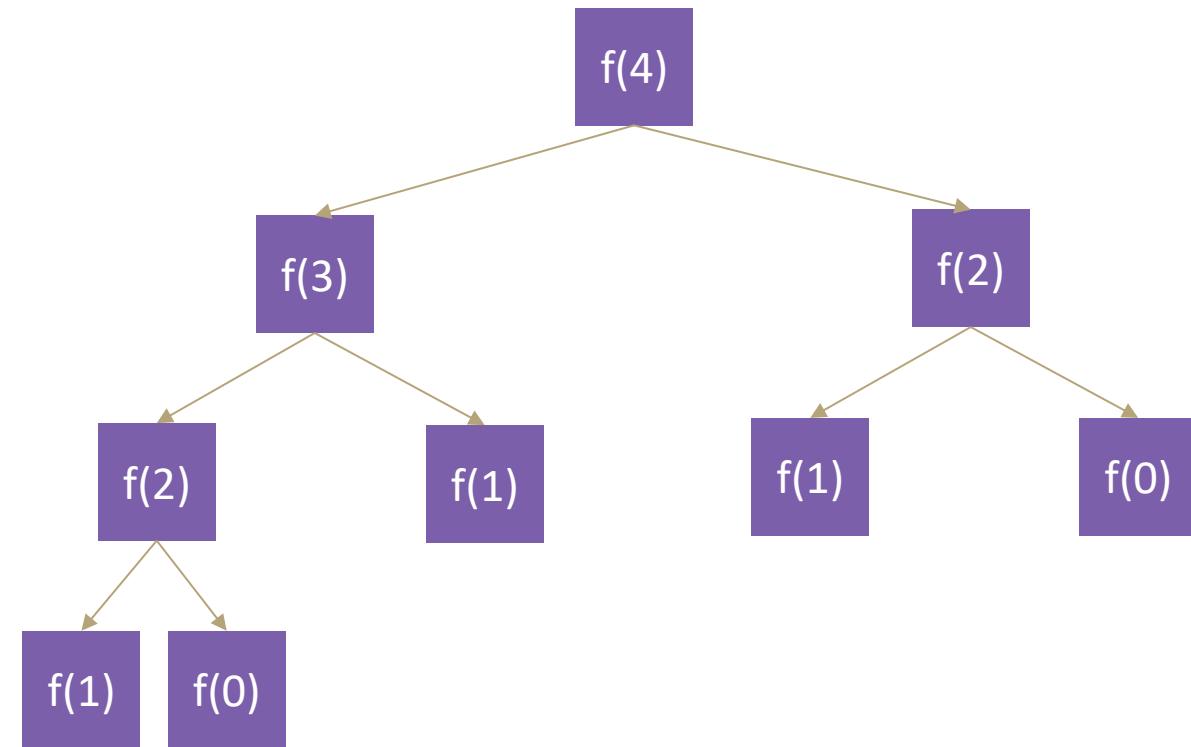
Wie viele Ebenen sind nötig, um „n“ durch Subtraktion von 1 in den Basisfall „1“ umzuwandeln?

Für unser Beispiel: 4 -> Höhe = n

Wie viele Funktionsaufrufe pro Ebene?

Wie viele Funktionsaufrufe auf Ebene k? 2^{k-1}

Ebene	Funktionsaufrufe
1	1
2	2
3	4
4	8



Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} d & \text{falls } n \leq 1 \\ T(n-1) + T(n-2) + 7 & \text{sonst} \end{cases}$$

Wie viele Ebenen enthält der Rekursionsbaum?

Wie viele Ebenen sind nötig, um „n“ durch Subtraktion von 1 in den Basisfall „1“ umzuwandeln?

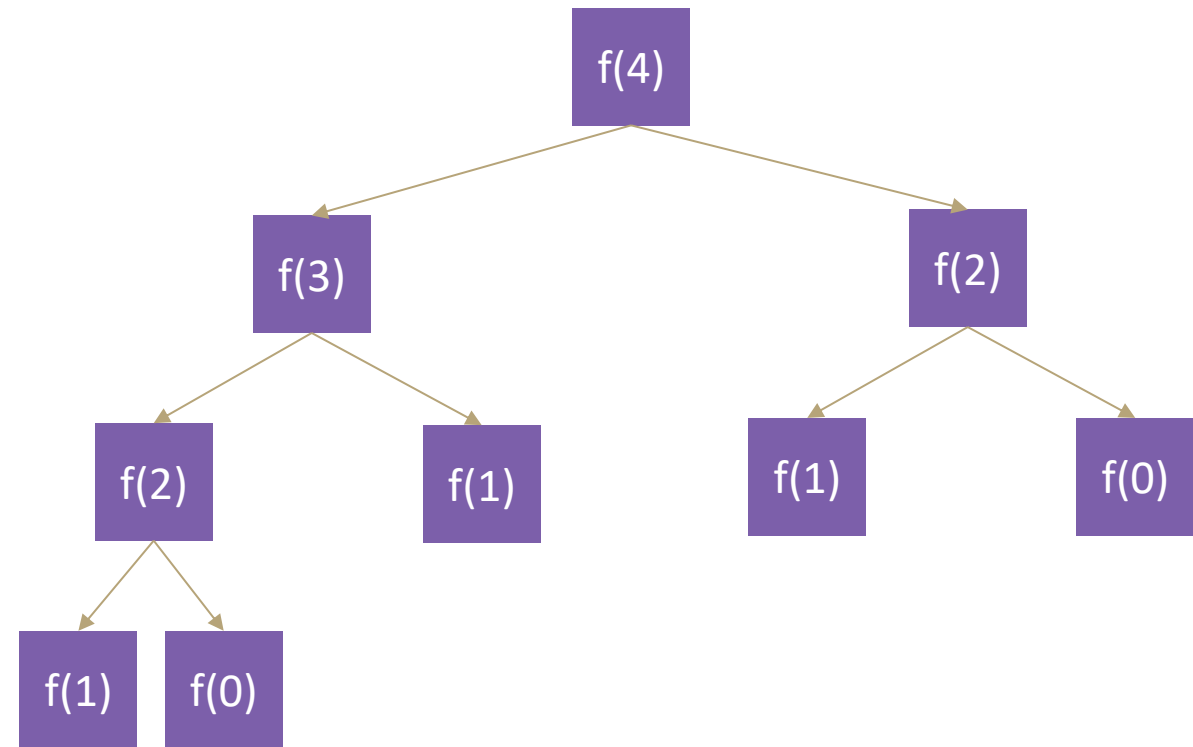
Für unser Beispiel: 4 -> Höhe = n

Wie viele Funktionsaufrufe pro Ebene?

Wie viele Funktionsaufrufe auf Ebene k? 2^{k-1}

Ebene	Funktionsaufrufe
1	1
2	2
3	4
4	8

Wie viele Funktionsaufrufe
GESAMT für einen Baum mit
n Ebenen?



Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} d & \text{falls } n \leq 1 \\ T(n-1) + T(n-2) + 7 & \text{sonst} \end{cases}$$

Wie viele Ebenen enthält der Rekursionsbaum?

Wie viele Ebenen sind nötig, um „n“ durch Subtraktion von 1 in den Basisfall „1“ umzuwandeln?

Für unser Beispiel: 4 -> Höhe = n

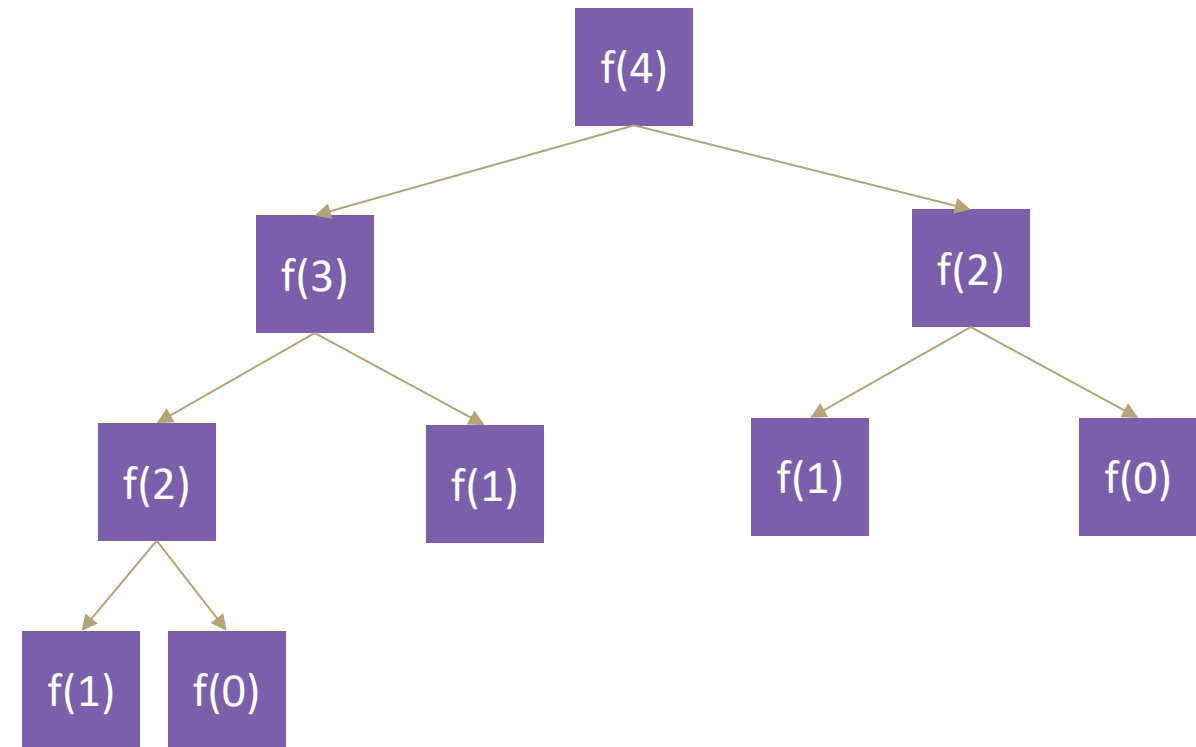
Wie viele Funktionsaufrufe pro Ebene?

Wie viele Funktionsaufrufe auf Ebene k? 2^{k-1}

Ebene	Funktionsaufrufe
1	1
2	2
3	4
4	8

Wie viele Funktionsaufrufe
GESAMT für einen Baum mit
n Ebenen?

$$1 + 2 + 4 + 8 + \dots + 2^{k-1} + \dots + 2^{n-1}$$



Berechnung der Fibonacci-Zahlen

Rekursionsgleichung in Θ -Notation umwandeln

$$T(n) = \begin{cases} d & \text{falls } n \leq 1 \\ T(n-1) + T(n-2) + 7 & \text{sonst} \end{cases}$$

Wie viele Ebenen enthält der Rekursionsbaum?

Wie viele Ebenen sind nötig, um „n“ durch Subtraktion von 1 in den Basisfall „1“ umzuwandeln?

Für unser Beispiel: 4 -> Höhe = n

Wie viele Funktionsaufrufe pro Ebene?

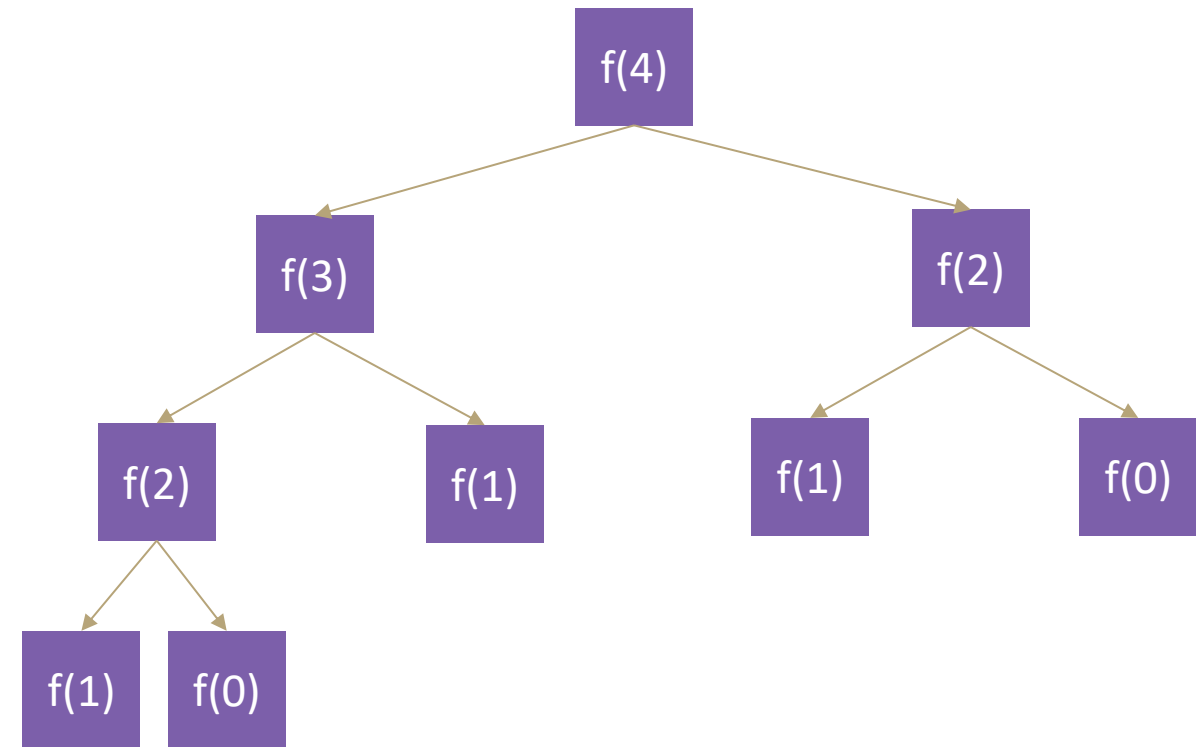
Wie viele Funktionsaufrufe auf Ebene k? 2^{k-1}

Ebene	Funktionsaufrufe
1	1
2	2
3	4
4	8

Wie viele Funktionsaufrufe
GESAMT für einen Baum mit
n Ebenen?

$$1 + 2 + 4 + 8 + \dots + 2^{k-1} + \dots + 2^{n-1}$$

$$T(n) \in \Theta(2^n)$$



Code Modellierung: Rekursive Muster

Muster Nr. 1: Halbierung der Eingabe

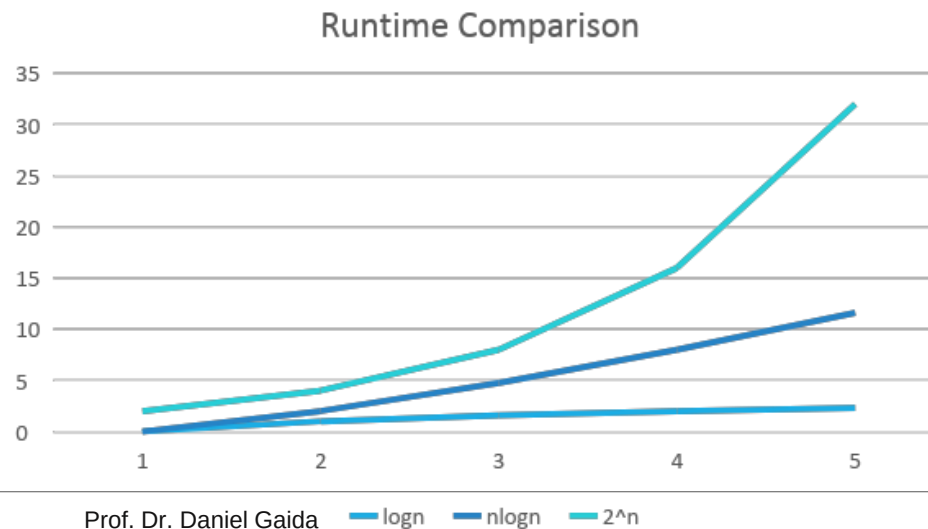
- Binäre Suche – Laufzeit $\Theta(\log n)$

Muster Nr. 2: Teilen der Eingabe, Beherrschen und Vereinigen

- Mergesort – Laufzeit $\Theta(n \log n)$

Muster Nr. 3: Verdoppelung der Eingabe

- Berechnung der Fibonacci-Zahlen – Laufzeit $\Theta(2^n)$



Code Modellierung: Rekursive Muster

Muster Nr. 1: Halbierung der Eingabe

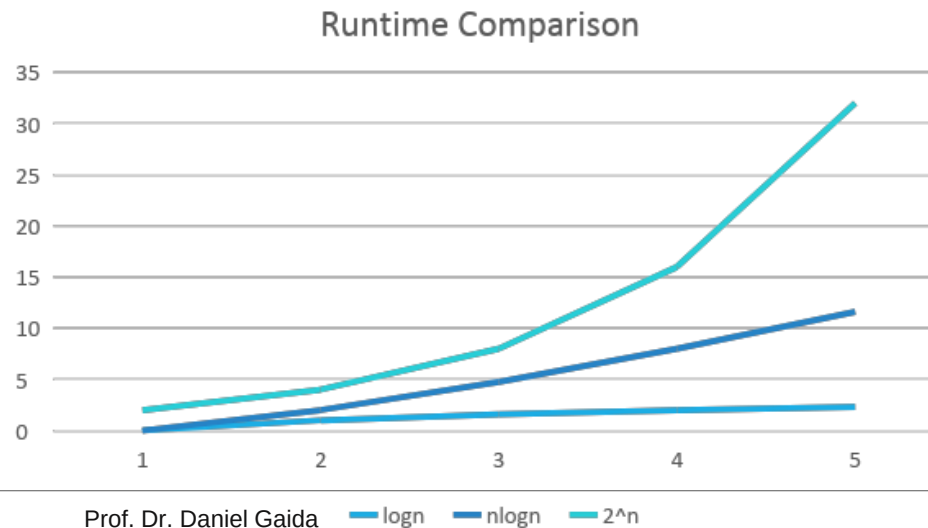
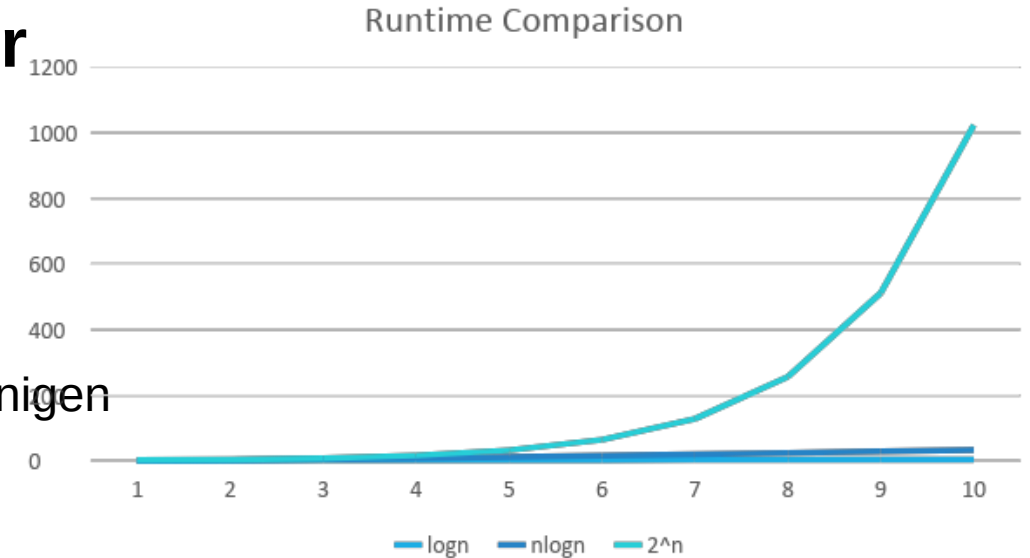
- Binäre Suche – Laufzeit $\Theta(\log n)$

Muster Nr. 2: Teilen der Eingabe, Beherrschen und Vereinigen

- Mergesort – Laufzeit $\Theta(n \log n)$

Muster Nr. 3: Verdoppelung der Eingabe

- Berechnung der Fibonacci-Zahlen – Laufzeit $\Theta(2^n)$



Code Modellierung: Rekursive Muster

Muster Nr. 1: Halbierung der Eingabe

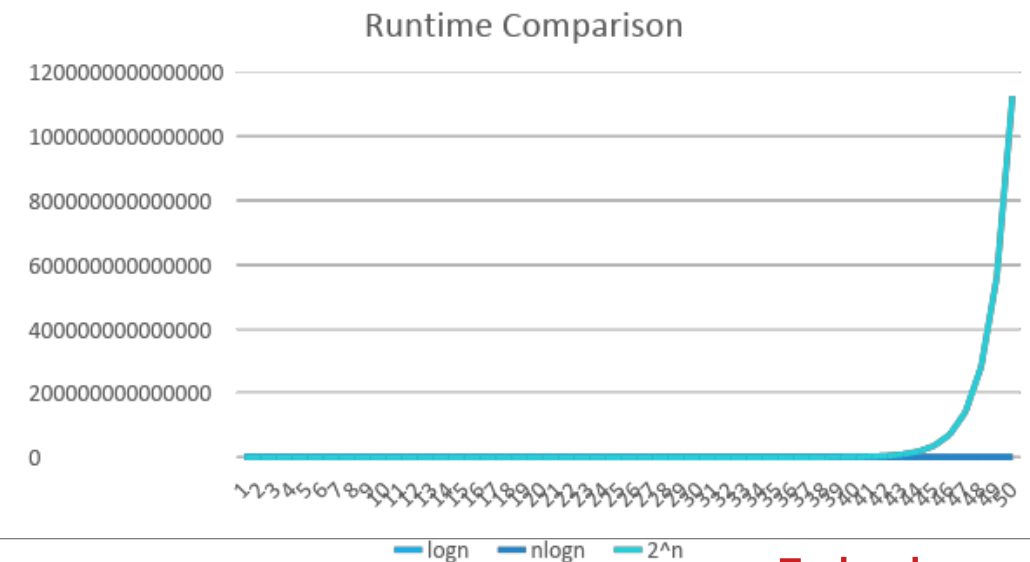
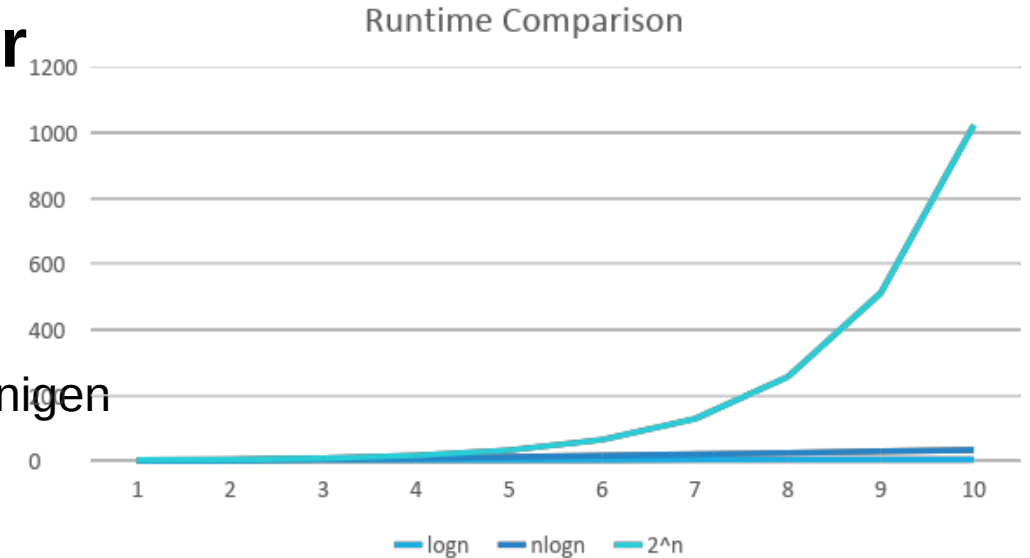
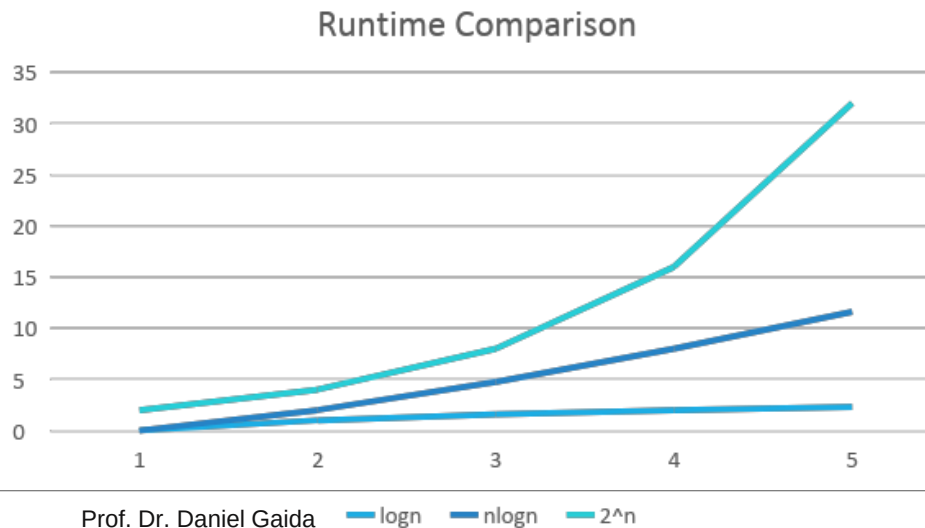
- Binäre Suche – Laufzeit $\Theta(\log n)$

Muster Nr. 2: Teilen der Eingabe, Beherrschen und Vereinigen

- Mergesort – Laufzeit $\Theta(n \log n)$

Muster Nr. 3: Verdoppelung der Eingabe

- Berechnung der Fibonacci-Zahlen – Laufzeit $\Theta(2^n)$



Lernziele von heute und Fragen zur Überprüfung der Lernziele

Die Studierenden können das asymptotische Laufzeitverhalten einiger rekursiver Algorithmen angeben, indem sie überprüfen, ob die Mastermethode anwendbar ist und diese dann anwenden, um später die Laufzeit einiger rekursiver Algorithmen bestimmen zu können.

s. Übungsblatt 3

Übungsblatt 3: Aufgabe 3, 7, 8, 9 und 10

Code Modellierung komplexer Schleifen

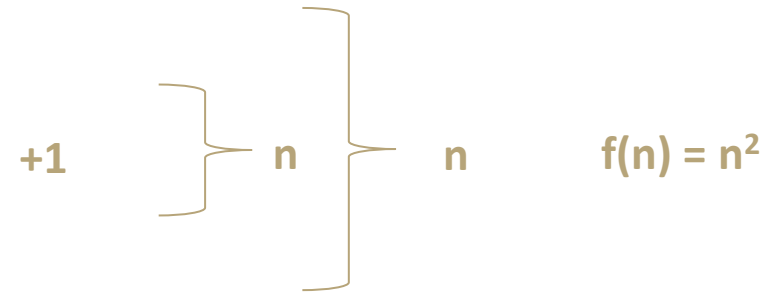
Schreiben Sie ein mathematisches Modell des folgenden Codes

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < i; j++) {  
        System.out.println("Hello!");  
    }  
}
```

Code Modellierung komplexer Schleifen

Schreiben Sie ein mathematisches Modell des folgenden Codes

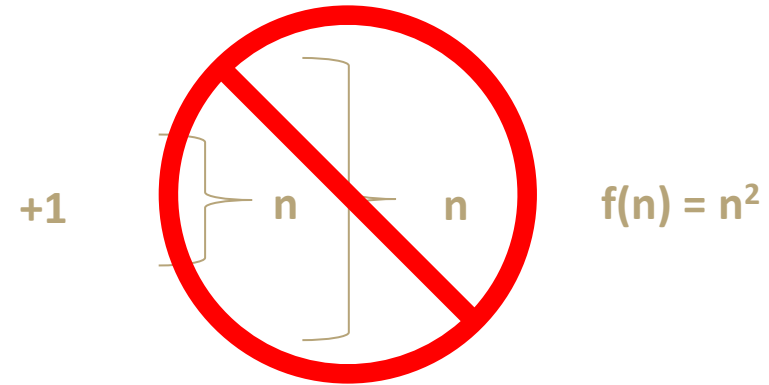
```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.println("Hello!");
    }
}
```



Code Modellierung komplexer Schleifen

Schreiben Sie ein mathematisches Modell des folgenden Codes

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.println("Hello!");
    }
}
```



Beachten Sie die Schleifengrenzen!

Code Modellierung komplexer Schleifen

Wie modellieren wir das?

```
for (int i = 1; i <= n; i++) {  
    System.out.println("Hello!");  
}
```

Code Modellierung komplexer Schleifen

Wie modellieren wir das?

```
for (int i = 1; i <= n; i++) {  
    System.out.println("Hello!");  
}
```

Summe!
 $f(n) = 1 + 1 + 1 + \dots + 1 =$

n-mal

Code Modellierung komplexer Schleifen

Wie modellieren wir das?

```
for (int i = 1; i <= n; i++) {  
    System.out.println("Hello!");  
}
```

Summe!

$$f(n) = \underbrace{1 + 1 + 1 + \dots + 1}_{n\text{-mal}} = \sum_{i=1}^n 1$$

Code Modellierung komplexer Schleifen

Wie modellieren wir das?

```
for (int j = 0; j < i; j++) {  
    System.out.println("Hello!");  
}
```

Code Modellierung komplexer Schleifen

Wie modellieren wir das?

```
for (int j = 0; j < i; j++) {  
    System.out.println("Hello!");  
}
```

Summe!

$$f(i) = \underbrace{1 + 1 + 1 + \dots + 1}_{i\text{-mal}} =$$

Code Modellierung komplexer Schleifen

Wie modellieren wir das?

```
for (int j = 0; j < i; j++) {  
    System.out.println("Hello!");  
}
```

Summe!

$$f(i) = \underbrace{1 + 1 + 1 + \dots + 1}_{i\text{-mal}} = \sum_{j=0}^{i-1} 1$$

Code Modellierung komplexer Schleifen

Wie modellieren wir das?

```
for (int j = 0; j < i; j++) {
    System.out.println("Hello!");
}
```

```
for (int i = a; i <= b; i++) {
    fu(i);
}
```

$fu(i) \in O(f(i))$

Summe!

$$f(i) = \underbrace{1 + 1 + 1 + \dots + 1}_{i\text{-mal}} = \sum_{j=0}^{i-1} 1$$

Definition: Summe

$$\sum_{i=a}^b f(i) = f(a) + f(a+1) + f(a+2) + \dots + f(b-2) + f(b-1) + f(b)$$

Code Modellierung komplexer Schleifen

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < i; j++) {  
        System.out.println("Hello!");  
    }  
}
```

Code Modellierung komplexer Schleifen

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.println("Hello!");
    }
}
```

$$f(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$

Code Modellierung komplexer Schleifen

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.println("Hello!");
    }
}
```

$$f(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$

Wie lautet die O-Notation?

Code Modellierung komplexer Schleifen

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < i; j++) {  
        System.out.println("Hello!");  
    }  
}
```



$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$

Code Modellierung komplexer Schleifen

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < i; j++) {  
        System.out.println("Hello!");  
    }  
}
```



$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$



geschlossene
Form

Code Modellierung komplexer Schleifen

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < i; j++) {  
        System.out.println("Hello!");  
    }  
}
```



$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$



geschlossene
Form



O-Notation

Code Modellierung komplexer Schleifen

Finden der geschlossenen Form unter
Verwendung von Rechenregeln für
Summen

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < i; j++) {  
        System.out.println("Hello!");  
    }  
}
```



$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$



geschlossene
Form



O-Notation

Code Modellierung komplexer Schleifen

Finden der geschlossenen Form unter
Verwendung von Rechenregeln für
Summen

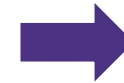
```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.println("Hello!");
    }
}
```



$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$



geschlossene
Form



O-Notation

$$f(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$

Code Modellierung komplexer Schleifen

Finden der geschlossenen Form unter Verwendung von Rechenregeln für Summen

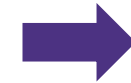
```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.println("Hello!");
    }
}
```



$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$



geschlossene Form



O-Notation

$$f(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=0}^{n-1} 1 \cdot i$$

Summe einer Konstanten

$$\sum_{i=0}^{n-1} c = cn$$

Code Modellierung komplexer Schleifen

Finden der geschlossenen Form unter Verwendung von Rechenregeln für Summen

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.println("Hello!");
    }
}
```



$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$



geschlossene Form



O-Notation

$$f(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=0}^{n-1} 1 \cdot i = 1 \sum_{i=0}^{n-1} i$$

Summe einer Konstanten

$$\sum_{i=0}^{n-1} c = cn$$

Konstante ausklammern

$$\sum_{i=a}^b cf(i) = c \sum_{i=a}^b f(i)$$

Code Modellierung komplexer Schleifen

Finden der geschlossenen Form unter Verwendung von Rechenregeln für Summen

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.println("Hello!");
    }
}
```



$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$



geschlossene Form



O-Notation

$$f(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=0}^{n-1} 1 \cdot i = 1 \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$$

Summe einer Konstanten

$$\sum_{i=0}^{n-1} c = cn$$

Konstante ausklammern

$$\sum_{i=a}^b cf(i) = c \sum_{i=a}^b f(i)$$

Gaußsche Summenformel

$$\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$$

Code Modellierung komplexer Schleifen

Finden der geschlossenen Form unter Verwendung von Rechenregeln für Summen

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.println("Hello!");
    }
}
```



$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$



geschlossene Form



O-Notation

$$f(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=0}^{n-1} 1 \cdot i = 1 \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$$

Summe einer Konstanten

$$\sum_{i=0}^{n-1} c = cn$$

Konstante ausklammern

$$\sum_{i=a}^b cf(i) = c \sum_{i=a}^b f(i)$$

Gaußsche Summenformel

$$\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$$

Code Modellierung komplexer Schleifen

Finden der geschlossenen Form unter Verwendung von Rechenregeln für Summen

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.println("Hello!");
    }
}
```



$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1$$



geschlossene Form



O-Notation

$$f(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=0}^{n-1} 1 \cdot i = 1 \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n \in O(n^2)$$

Summe einer Konstanten

$$\sum_{i=0}^{n-1} c = cn$$

Konstante ausklammern

$$\sum_{i=a}^b cf(i) = c \sum_{i=a}^b f(i)$$

Gaußsche Summenformel

$$\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$$

Code Modellierung komplexer Schleifen

Übungsblatt 2: Aufgabe 2, 13, 5, 8, 9, 11

Übungsblatt 3: Aufgabe 2, 11, 5, 1

Nächste Vorlesung und Fragen

- Lernraum II: Höhere Abstrakte Datentypen und Datenstrukturen
 - Hashing, Datenstruktur: Hashtabelle
- Fragen?